Programare Python

2024 - 2025

Sorin Milutinovici
sorin.milutinovici@prof.utm.ro

Organizarea cursului

Cursul de "Programare Python" este compus dintr-o prelegere și 4 sesiuni de lucru.

Prelegerea trece în revista diferențele dintre limbajele compilate și cele interpretate, prezinta extensii moderne ale paradigmei procedurale si vorbeste pe scurt despre limbajul Python.

Sesiunile de lucru se vor desfășura cu ajutorul Jupyter. Jupyter (Julia, Python, R) este o consolă interactivă în care se pot introduce succesiv blocuri de cod și text și se pot executa. Ideal ar fi ca studenții sa urmareasca în propriul lor notebook jupyter ceea ce se scrie astfel incat sa putem rezolva eventualele erori sau neînțelegeri în timp real.

Jupyter se poate instala local sau se poate utiliza online. Exista mai multe platforme care permit rularea python in Jupyter:

Google Colaboratory Jupyter lab Kaggle	GoCalc JetBrains Datalore Saturncloud

Pentru instalarea locală, exista resurse online:
Windows: https://sparkbyexamples.com/python/install-anaconda-jupyter-notebook/
Ubuntu: https://medium.datadriveninvestor.com/how-to-install-the-jupyter-notebook-server-on-linux-b2c14c47b446

Cursul se va încheia cu un examen grila compus din 20 de întrebări.

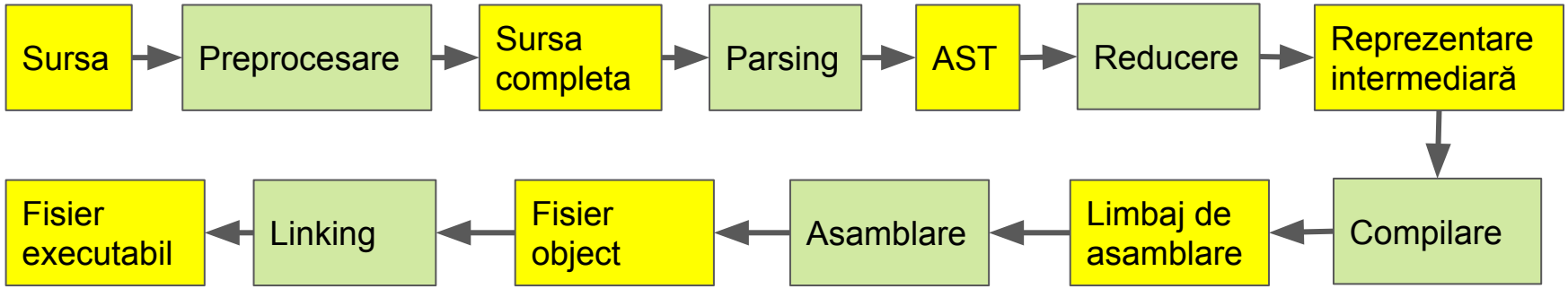
Program, process, sistem de operare

Mașini virtuale de proces

Extinderi ale paradigmei procedurale

Limbajul de programare Python

Cum ajunge un program sa ruleze într-un sistem de operare?



Compilare

- Compilare înseamnă transformarea unui program scris într-un anumit limbaj într-un program scris în alt limbaj. Termenul este folosit în sens strict pentru a desemna programele care transformă din cod sursă în cod mașină.

- Compilatoarele sunt sisteme de programe formate din mai multe componente. Majoritatea lor pot obține cod binar pentru mai multe arhitecturi și pentru mai multe sisteme de operare. În general, un compilator se considera a fi alcătuit din trei grupuri de componente:

- **Front-end**

  Grupul front-end reprezintă componente dedicate limbajului de intrare. Acestea sunt special concepute pentru a analiza și corecta sintaxa programului care va fi compilat și pentru a-l transforma într-o reprezentare intermediară - un fel de limbaj generic, numit de multe ori AST - Abstract Syntax Tree, care se transformă apoi prin reducere (derularea expresiilor imbricate, variabile temporare, control flow explicit) în reprezentarea intermediară.

- **Middle-end**

  Componentele din grupul Middle-end optimizează ceea ce primesc - reprezentarea intermediară. Componentele din middle end nu cunosc limbaje de programare și nu cunosc detalii despre hardware, acestea operează cu structuri arborescente de date și instrucțiuni abstracte.

- **Back-end**

  Componentele din grupul back-end generează în final codul executabil al programului. Componentele din back-end știu să vorbească limba procesorului și a sistemului de operare pentru care se compilează programul respectiv.

Compilare - front-end

- Operațiile prin care trece codul sursă în zona front-end depind de limbajul de programare (front-end este specifică limbajului) și de implementare. De cele mai multe ori acestea sunt următoarele:

- **Preprocesarea** - este o etapa de completare a codului sursa. Multe sisteme de compilare permit refolosirea codului, sau a declarațiilor acestuia, de exemplu prin diverse instrucțiuni de includere.

- **Analiza lexicală** - convertește textul programului într-o succesiune de fragmente care au sens pentru compilator. Dacă este identificat un cuvânt care nu este “înțeles”, procesul se oprește cu o eroare.

- **Analiza sintactică** - Fragmentele identificate în etapa anterioară sunt așezate într-o structură arborescentă conform regulilor gramaticale ale limbajului respectiv.

- **Analiza semantică** - Structura arborescentă este îmbogățită cu informații despre semnificația fiecărui nod. Se construiește tabelul de simboluri (un index al tuturor variabilelor/functiilor/claselor,,, etc.).

Preprocesare

Preprocesarea presupune alcătuirea unui cod complet care să includă și declarațiile externe și să expandeze toate macro-urile specificate în cod.

```
#include <stdio.h>

int main()
{
   printf("Hello, World!");
   return 0;
}
```

```
gcc -Wall hello.c -o hello
```

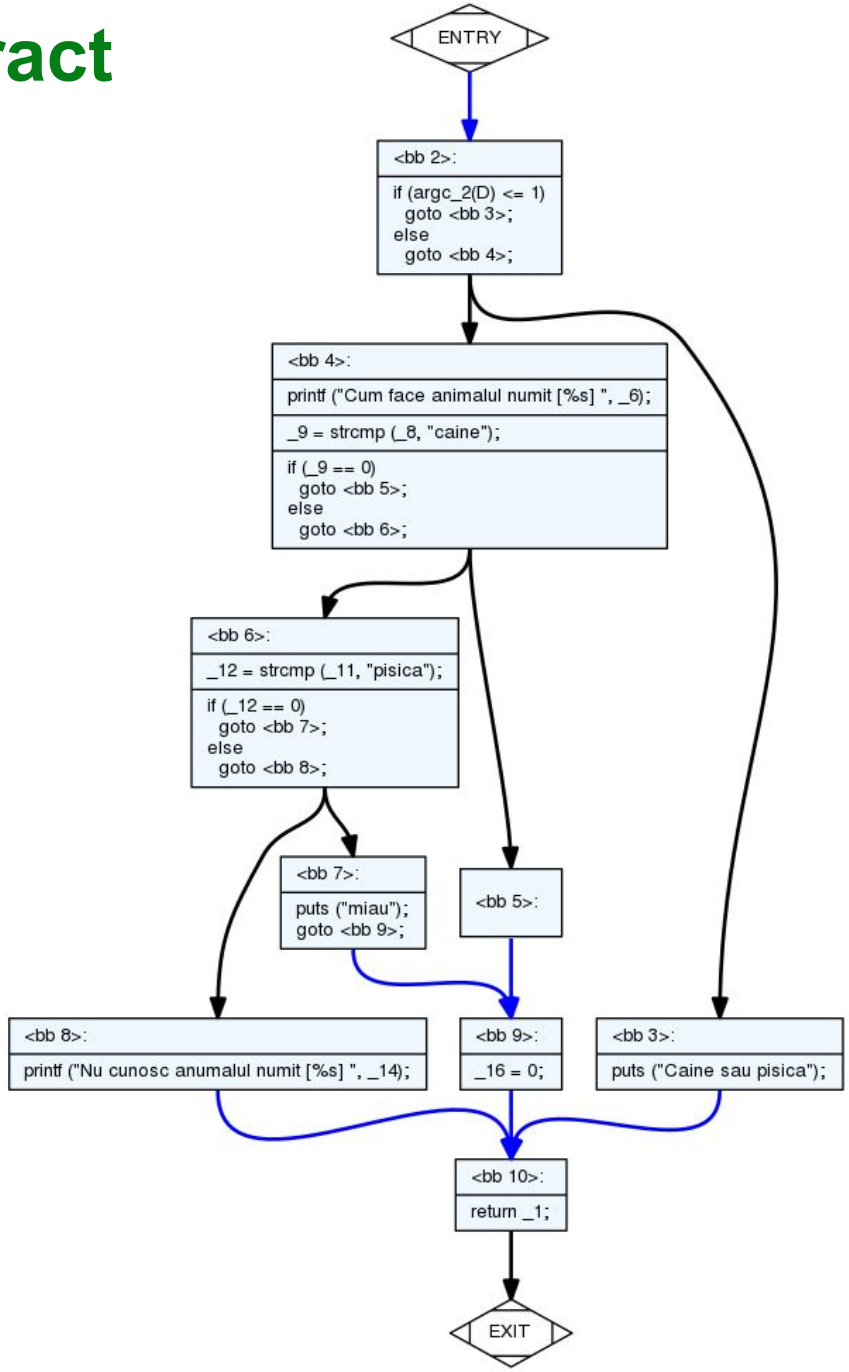
```
gcc -E hello.c
```

```
# 1 "hello.c"
# 1 "<built-in>"
# 1 "<command-line>"
--------------fragment-----------------
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 1 "<command-line>" 2
# 1 "hello.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 27 "/usr/include/stdio.h" 3 4

extern int fprintf (FILE *__restrict __stream,
      const char *__restrict __format, ...);

extern int printf (const char *__restrict
__format, ...);

extern int sprintf (char *__restrict __s,
      const char *__restrict __format, ...)
__attribute__ ((__nothrow__));

--------fragment------------

# 2 "hello.c" 2

int main()
{
   printf("Hello, World!");
   return 0;
}
```

Limbaj intermediar (Abstract Syntax Tree)

```
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[])
{
    if (argc < 2) {
        puts ("Caine sau pisica");
        return 0;
    } else {
        printf("Cum face animalul numit [%s] \n", argv[1]);
        if (strcmp(argv[1], "caine") == 0) {
            puts ("ham");
        } else if (strcmp(argv[1], "pisica") == 0) {
            puts ("miau");
        } else {
            printf ("Nu cunosc animalul numit [%s] \n", argv[1]);
            return 0;
        }
            return 0;
    }
}
```

https://onlinegdb.com/aON0J0mOE
(este nevoie de argument caine sau pisica la rulare)



Back-end

- Backend-ul poate aplica propriile optimizări, dependente de arhitectura vizată.
- Faza de compilare produce un cod în limbaj de asamblare.
- După faza de compilare urmează faza de asamblare, în care codul este modificat în echivalentele binare ale comenzilor.
- Faza de asamblare produce un **fișier obiect.**

```
gcc -s hello.c
```

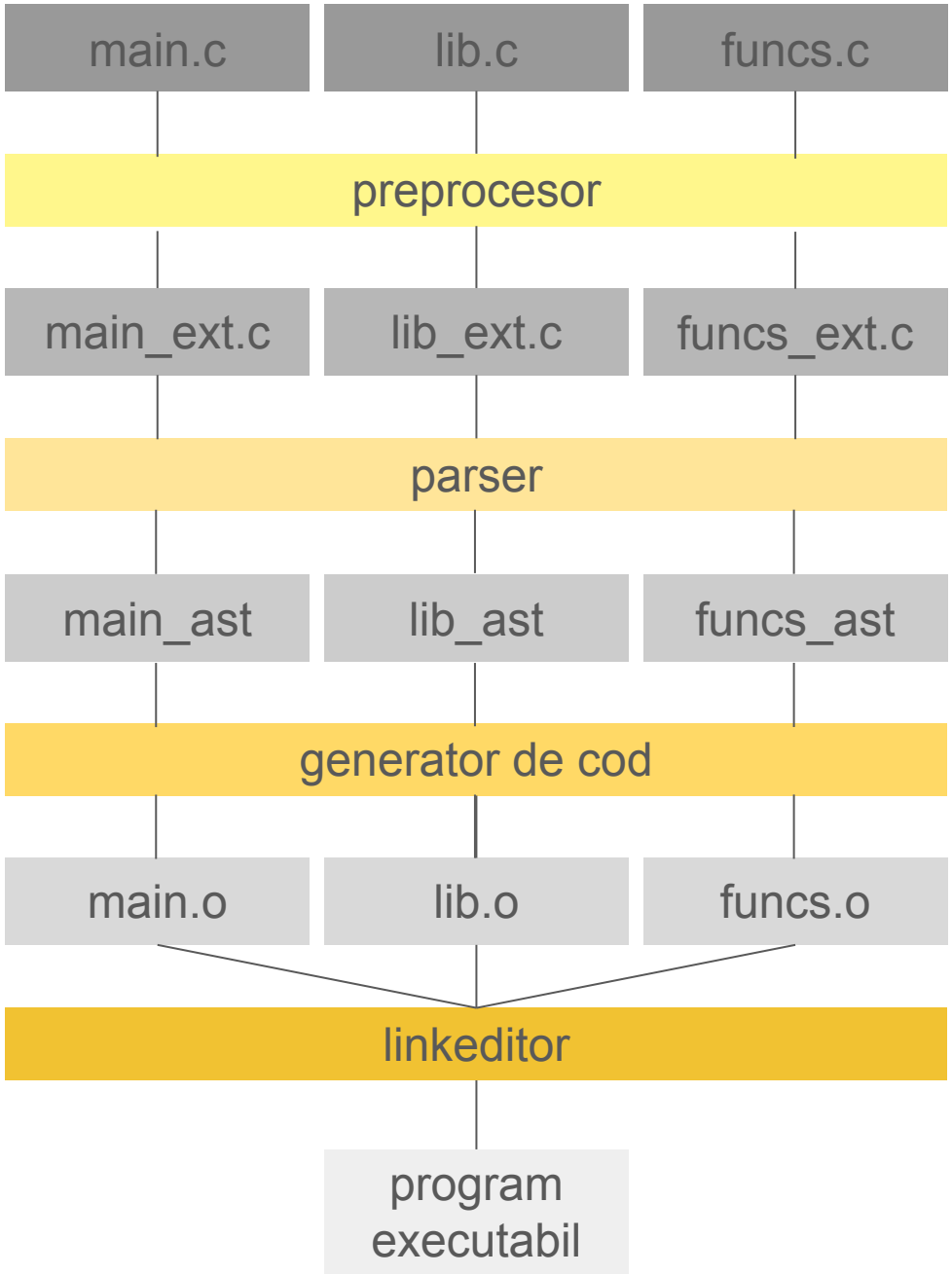
```
    .file   "hello.c"
    .intel_syntax noprefix
    .section .rodata
.LC0:
    .string "Hello, World!"
    .text
    .globl  main
    .type   main, @function
main:
.LFB0:
    .cfi_startproc
    push rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    mov rbp, rsp
    .cfi_def_cfa_register 6
    mov edi, OFFSET FLAT:.LC0
    mov eax, 0
    call printf
    mov eax, 0
    pop rbp
    .cfi_def_cfa 7, 8
    ret
    .cfi_endproc
.LFE0:
    .size   main, .-main
    .ident  "GCC: (SUSE Linux) 4.8.5"
    .section .note.GNU-stack,"",@progbits
```

Fișier obiect?

Cele mai multe programe compilate depind de o serie de alte biblioteci care sunt specificate în codul sursă dar nu apar în fișierul de tip obiect. Pentru a obține un program executabil, fișierul de tip obiect trebuie să fie “legat” de celelalte biblioteci necesare.

Linking:

1. Caută în fișierul obiect referințe către biblioteci externe care vor trebui încărcate în viitorul fișier executabil.
2. După ce determină toate simbolurile interne și externe necesare programului, linker-ul rezolvă referințele dintre ele și ajustează (relochează) adresele în cod și date astfel încât fiecare referință să indice locația corectă în executabil
3. Construiește executabilul final.



Executarea programului

- Un program este un fișier binar care are un format pe care atât nucleul sistemului de operare cât și procesorul îl pot înțelege.
- Nucleul sistemului de operare este responsabil de “încărcarea” programului în memorie - încărcare care implică maparea anumitor zone ale programului în memoria virtuală a procesului respectiv.
- Printre informațiile încărcate de nucleul sistemului de operare în memorie se află și seturi de instrucțiuni destinate procesorului. Acestea trebuie să fie compatibile cu setul de instrucțiuni pe care procesorul le poate executa.
- Procesorul va începe să execute instrucțiunile încărcate de nucleu în memorie, una câte una, până când acestea se vor termina, apoi procesul se va închide și memoria va fi eliberată.
- **Pentru ca un program să poată fi transformat într-un proces trebuie ca el să fie în același timp compatibil cu nucleul sistemului de operare și cu procesorul pe care acesta rulează.**

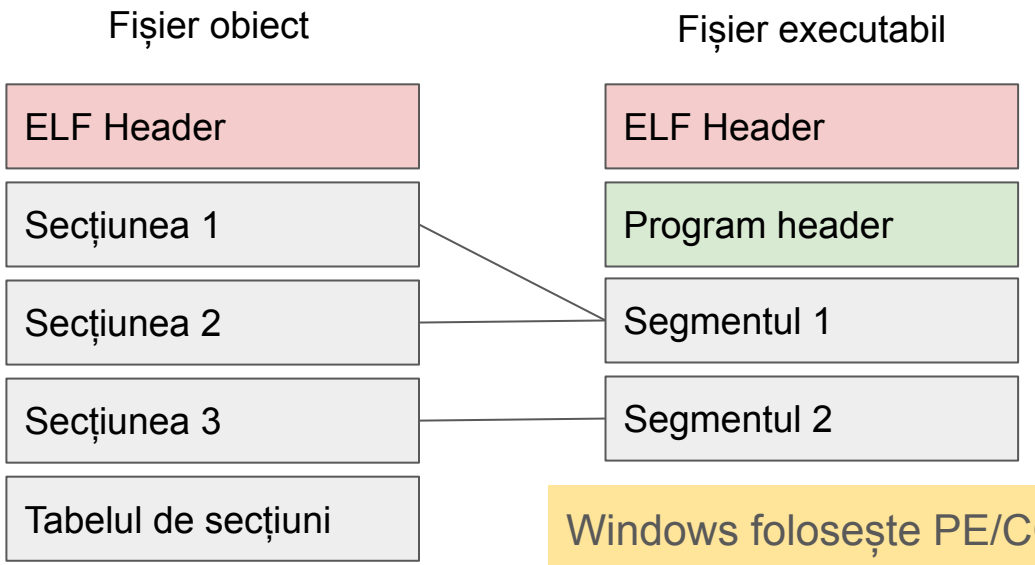
ELF (Executable and Linkable Format, Linux)

ELF poate fi folosit la următoarele tipuri de fișiere:

- **Fișiere executabile** - programe care sunt încărcate de către sistemul de operare și trimise spre procesare
- **Fișiere obiect** - seturi de instrucțiuni care vor fi incluse ulterior în fișierele executabile
- **Biblioteci partajate** - colecții de seturi de instrucțiuni care pot fi incluse în programe în timpul încărcării lor
- **Core dump** - înregistrarea conținutului memoriei atribuite unui anumit proces, de regulă la apariția unei erori.

Compilarea unui program produce de obicei fișiere obiect. Fișierele executabile se obțin printr-un proces ulterior, numit linking, care finalizează procesul, încorporând secțiunile fișierelor obiect în segmentele fișierului executabil.

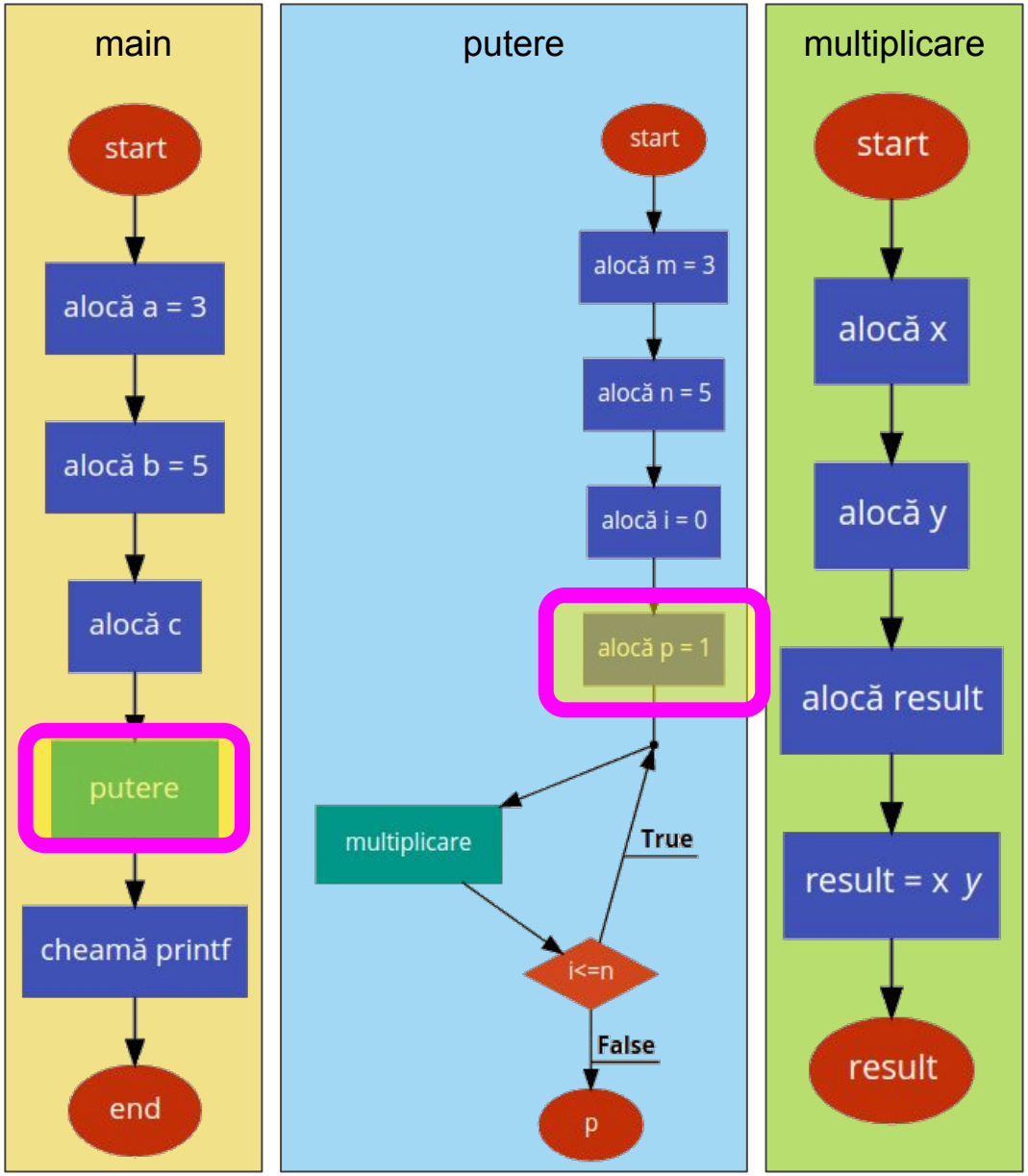
Secțiunile se interpretează de către un program special și sunt transformate în segmente care pot fi încărcate de nucleu.

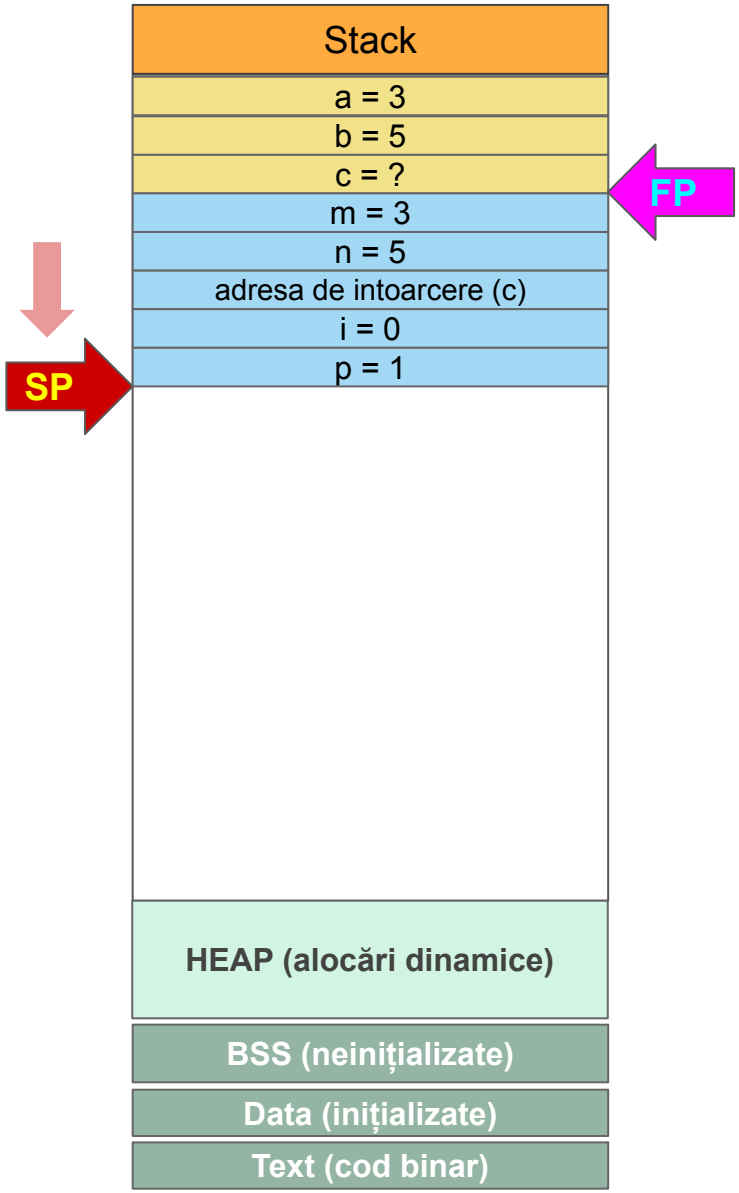


Segmentele se încarcă în memorie de către nucleul sistemului de operare și seturile de instrucțiuni din ele se execută de către procesor.

Windows folosește PE/COFF ca formate executabile

Memoria în timpul executării





Încărcarea și executarea programelor în sistemele de operare

Programele pe care le scriem și le compilăm sau le lansăm în execuție cu ajutorul interpretoarelor, sunt în final executate de un sistem de operare.

Chiar și în cazul execuției în cloud, chiar și în modelul "serverless computing", undeva există un sistem de operare care execută un program.

Un moment crucial în evoluția sistemelor de operare a fost introducerea conceptului de time-sharing, precursorul multitasking-ului modern. Deși Christopher Strachey a publicat o lucrare teoretică fundamentală despre acest subiect în 1959 (nu 1963), John McCarthy este cel care a definit și promovat viziunea acestui sistem la MIT încă din aceeași perioadă, ducând ulterior la prima implementare practică (CTSS, coordonată de Fernando Corbató). McCarthy rămâne o figură monumentală în informatică, fiind cel care a dat numele domeniului Inteligenței Artificiale, a creat limbajul LISP și a inventat conceptul de garbage collection, transformând fundamental modul în care gestionăm memoria și interacțiunea cu procesorul..

Time-sharing permitea unui sistem de operare să suspende execuția unui program (fără să-l oprească), să se mute la alt program, urmând ca apoi să revină la primul și să îl poată executa din punctul de unde a rămas. La o viteză acceptabilă de execuție, această mutare foarte rapidă dă senzația execuției paralele a programelor multiple în același timp.

Sistemele de operare fac aceasta de multă vreme. Recent, motoarele de execuție a anumitor medii de programare (ex: Mașina virtuală Java, executoarele PhP, Python, anumite implementări javascript) au început să permită modele de execuție similare permițând extinderea paradigmei procedurale.

Transformarea unui program într-un proces

Transformarea unui program într-un proces se face de către un component special al sistemului de operare, numit “loader”.

- Verifică integritatea și compatibilitatea formatului executabil (arhitectura corecta, programul e complet, etc.)
- Solicită altor componente ale sistemului de operare verificarea drepturilor de execuție
- Mapează imaginea programului în memoria principală a sistemului
- Mapează în zone speciale de memorie funcțiile necesare din biblioteci dinamice (dacă este cazul)
- Copiază datele de intrare ale programului (ex: argumente date la linia de comandă) în stack
- Inițializează registrele procesorului
- Pornește **execuția funcției principale** a programului

Funcția principală (main)

Majoritatea formatelor executabile moderne au un singur punct de intrare caruia ii este cedat controlul după încărcarea programului în memorie

Anumite limbaje de programare solicita precizarea explicită a punctului de intrare (C, C++, Java, etc.) iar altele, în special limbajele interpretate, ghicesc punctul de intrare prin utilizarea primei declarații executabile din corpul programului care nu se găsește în definiția unei funcții (perl, php, javascript, ruby, etc.)

```c
#include <stdio.h>

int main() {
   int n = 4, i;
   float num[] = {12.32, 16.21, 10, 18.1};
   float sum = 0.0;
   float avg;

   for (i = 0; i < n; ++i) {
       sum += num[i];
   }

   avg = sum / n;
   printf("Media = %.2f", avg);
   return 0;
}
```

main_func.c

https://onlinegdb.com/eqNcFP7dn

main

```php
<?php

$array = array(12.32, 16.21, 10, 18.1);
$sum = 0;
for ($i = 0; $i < count($array); ++$i) {
   $sum += $array[$i];
}

$average = $sum / count($array);



echo $average;
```

main_func1.php

https://onlinegdb.com/0D85YHdDt

Php oferă funcții specifice pentru matrici care permit scrierea programului într-o variantă mult redusă

```php
<?php

$array = array(12.32, 16.21, 10, 18.1);

$average = array_sum($array)/count($array);

echo $average;
```

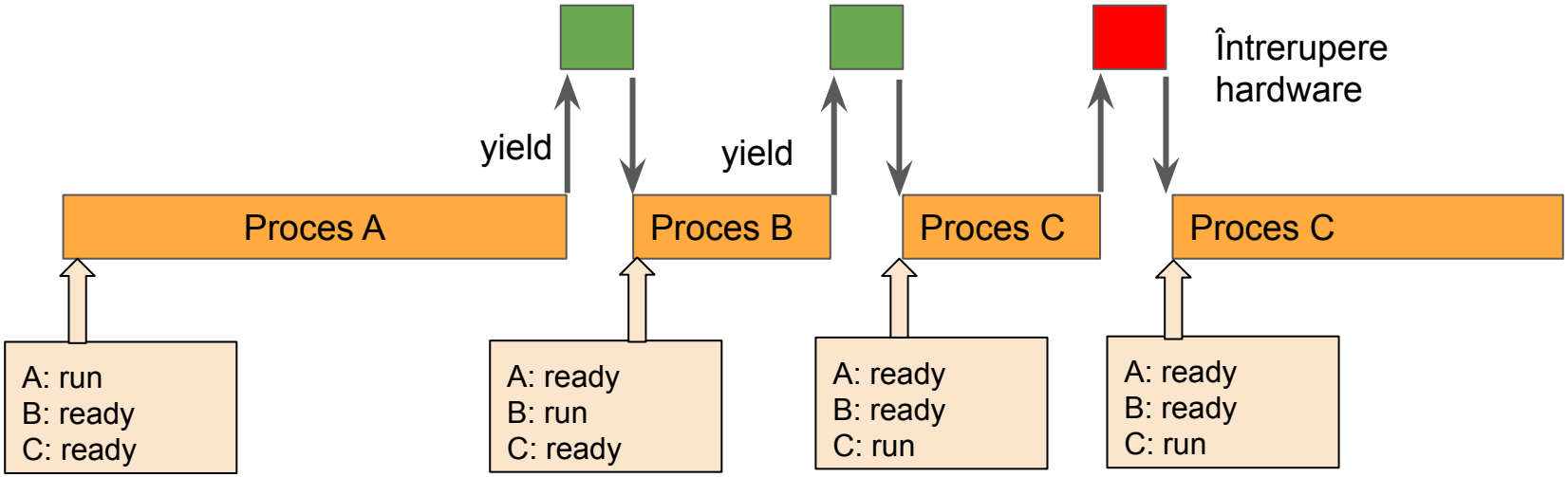
main_func2.php

https://onlinegdb.com/OdKMpx2QaI

Multitasking

Toate sistemele de operare moderne rulează mai multe programe în același timp. Această caracteristică se numește “multitasking”. Multitaskingul implică suspendarea temporară a execuției proceselor cu permiterea rulării unuia dintre ele. Fiecare proces care se află în memorie este rulat din cand in cand. “Semnalul” care declanșează mutarea între procese este dat de întreruperile hardware, provenite de obicei de la ceasul plăcii de bază. Întreruperile pot fi de mai multe tipuri și **au câte o procedură asociată** - această procedură nu face parte din structura vreunui program ci este o succesiune de instrucțiuni predefinite care se execută de câte ori apare acel tip de întrerupere.

Multitasking cooperativ - fiecare proces cedează de bună voie controlul către sistemul de operare

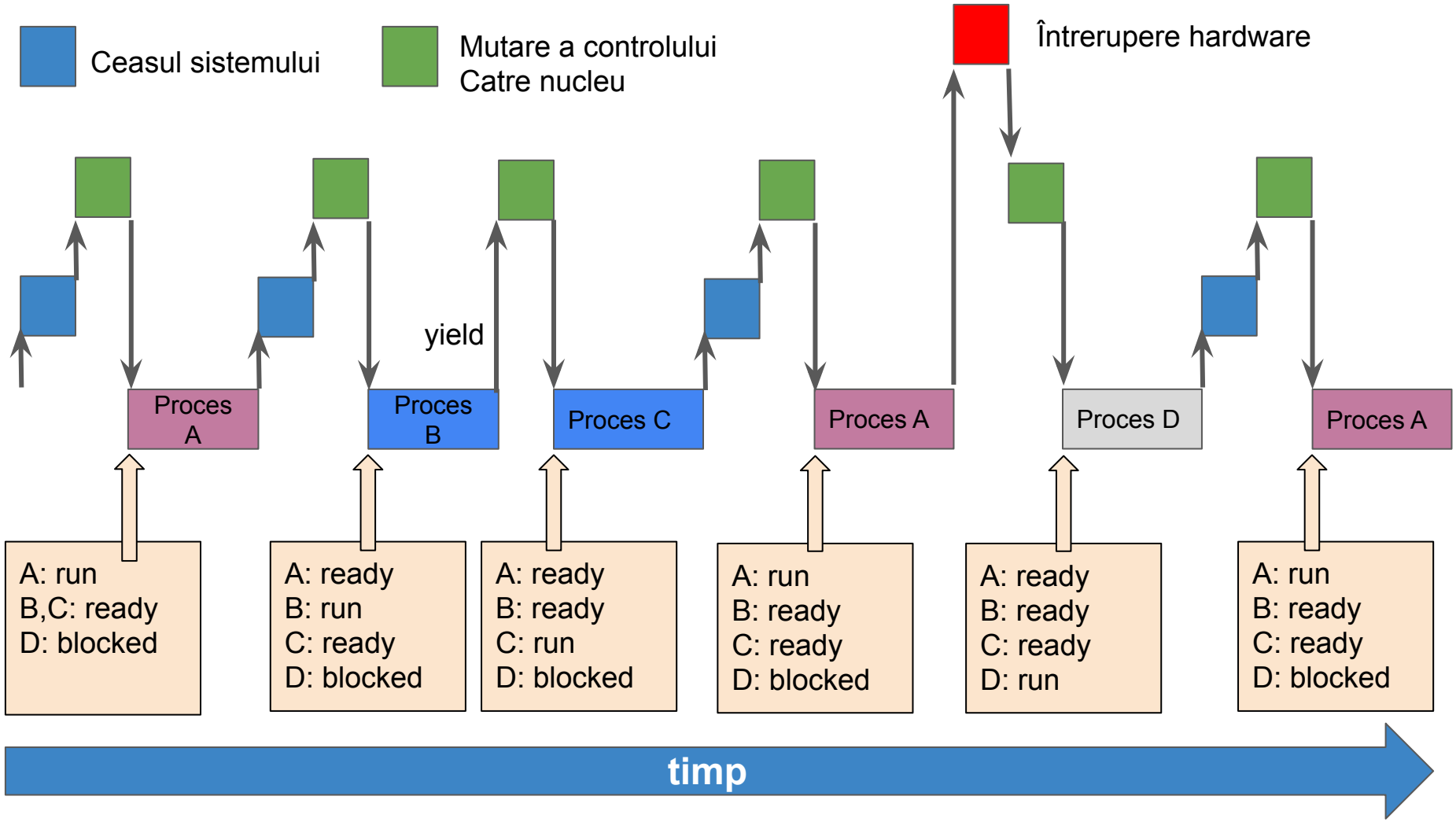


Windows 3.1

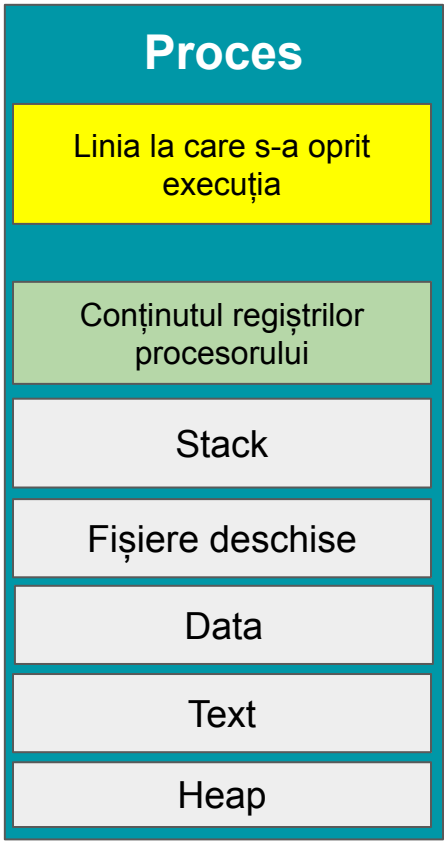
Mac OS clasic

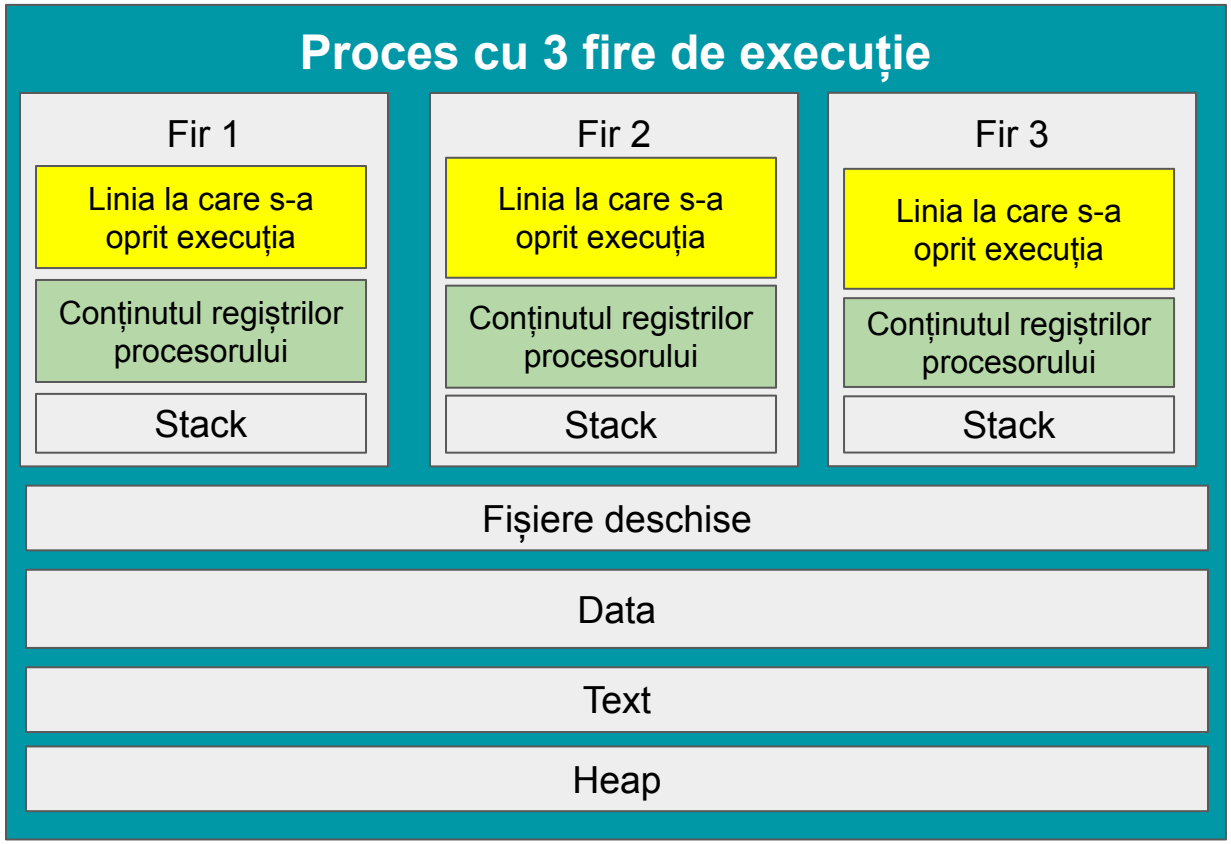
Multitasking preemptiv

Multitasking-ul preemptiv forțează suspendarea execuției proceselor și trecerea controlului către nucleu. Procesele pot în continuare să cedeze controlul voluntar dar, dacă nu o fac, vor fi suspendate după un timp predefinit de către algoritmul de planificare



Fire de execuție (threads) și procese





Procese - instanțe de execuție cu absolut toate segmentele de memorie izolate.
Thread-uri - procese care au mai multe fire de execuție prin separarea stivelor și a conținutului registrelor. Procesul părinte se ocupă de suspendarea/pornirea firelor de execuție. Un proces cu mai multe fire de execuție poate fi considerat un fel de mașină virtuală.

Multitaskingul din interiorul unui proces cu mai multe fire de execuție este de obicei preemptiv. Dacă acesta este cooperativ atunci firele de execuție se mai numesc “**fibre**” (FIBERS).

Un proces (sau un thread) are un singur stack deci poate executa o singură procedură la orice moment!

Un procesor (nucleu) are o singură linie de execuție deci poate executa un singur proces la orice moment!

Un procesor poate optimiza execuția proceselor prin multitasking (cooperativ sau preemptiv). Optimizarea este reală - procesele care așteaptă date sunt puse în așteptare. În final se obține o execuție mai fluidă și mai eficientă a aceleiași liste de procese.

Putem face ceva similar multitaskingului în programare?

- Putem defini proceduri care să se execute nu prin apelare directă ci ca răspuns la un anumit eveniment, asociind un set de instrucțiuni acelui eveniment (callbacks, closures).
- Putem construi proceduri care să cedeze controlul în timpul execuției (yield) urmând să fie “suspendate” până la următorul apel și să își reia execuția de unde au rămas (generatori).
- Putem construi mai multe stack-uri care să constituie fire distincte de execuție în interiorul aceluiași program. În continuare într-un moment dat va exista un singur stack “activ” dar altele pot fi suspendate și reluate atunci când este nevoie (corutine)

Dar modelul de execuție impus de hardware nu permite aceste lucruri

Nu toate programele se compilează direct pentru procesor!!

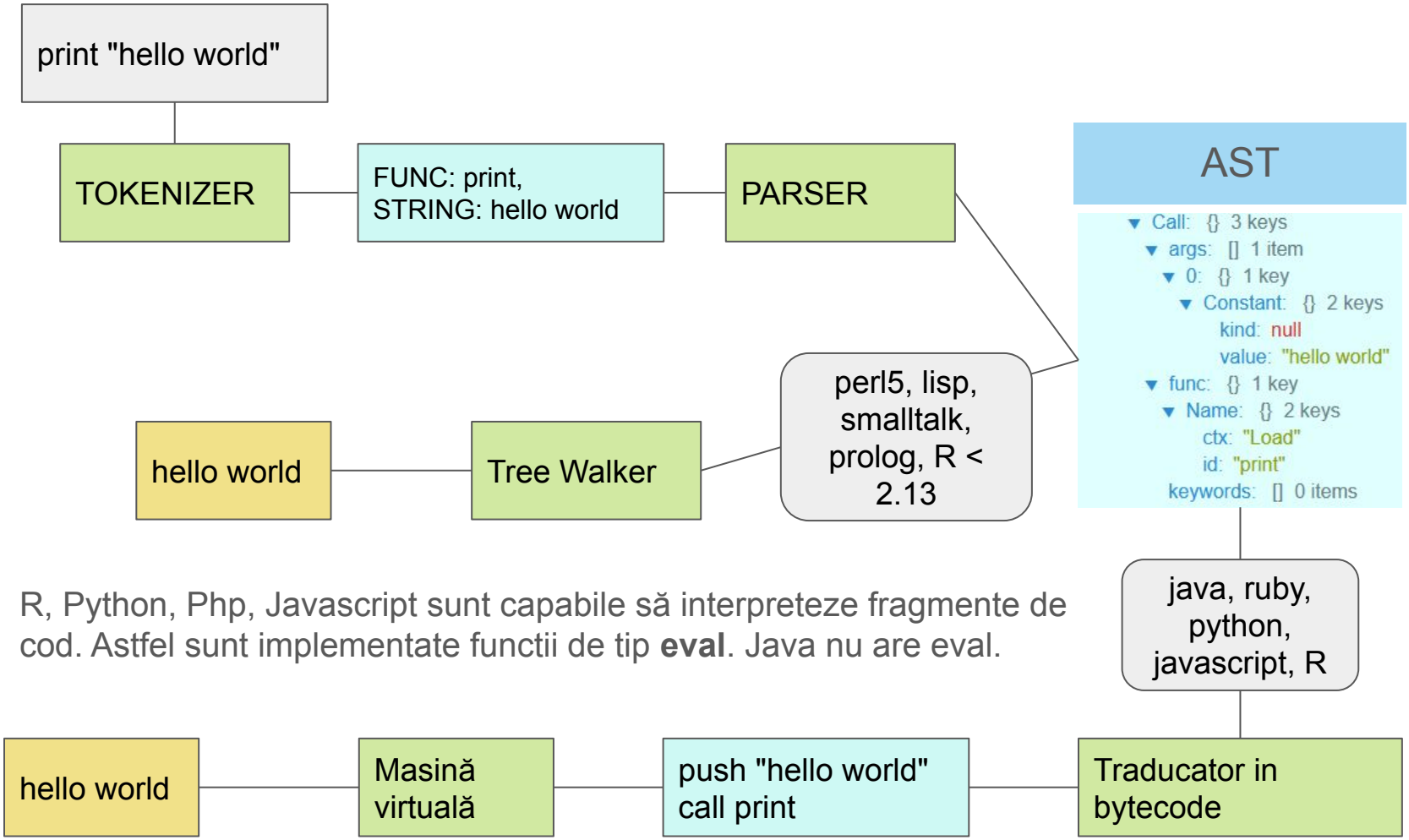
O mare parte din limbajele de programare noi compilează sursele generând ceea ce se numește "bytecode" - un fel de stream binar care are instrucțiuni pentru un procesor virtual, un CPU software emulat de o masina virtuala (process virtual machines, BCPL, cca 1966, apoi p-code - Pascal, 1970):

Java,Python,Php,Javascript,Ruby,Pascal,Visual Basic, etc.

Acestea sunt mașini virtuale "complete" - implementează un procesor virtual împreună cu un mic sistem de execuție virtual, asemănător unui sistem de operare care are propriul planificator de execuție, fire de execuție și management al memoriei.

Mașinile virtuale permit detașarea rulării programului de suportul hardware (stack, regiștrii). Această detașare face relativ ușoară implementarea unor concepte care **extind paradigma programării procedurale**.

Mașini virtuale de proces



R, Python, Php, Javascript sunt capabile să interpreteze fragmente de cod. Astfel sunt implementate functii de tip **eval**. Java nu are eval.

Pe langa traiectoria de transformare și interpretare a instrucțiunilor motoarele de execuție conțin și elemente de management al memoriei, garbage collectors, event loops, etc.

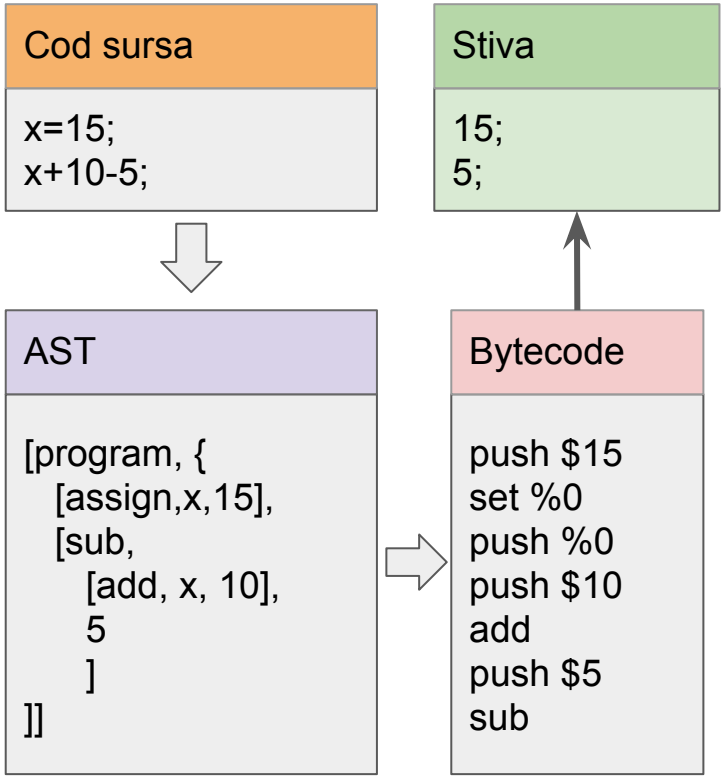
Mașini virtuale de proces

Tip Stiva (Stack based)

Java, Python, Javascript, etc.

Mașinile virtuale de tip stivă utilizează o stivă pentru a stoca operatorii și datele. Acestea sunt introduse și scoase din stivă

Rezultatul operației este pe prima poziție a stivei

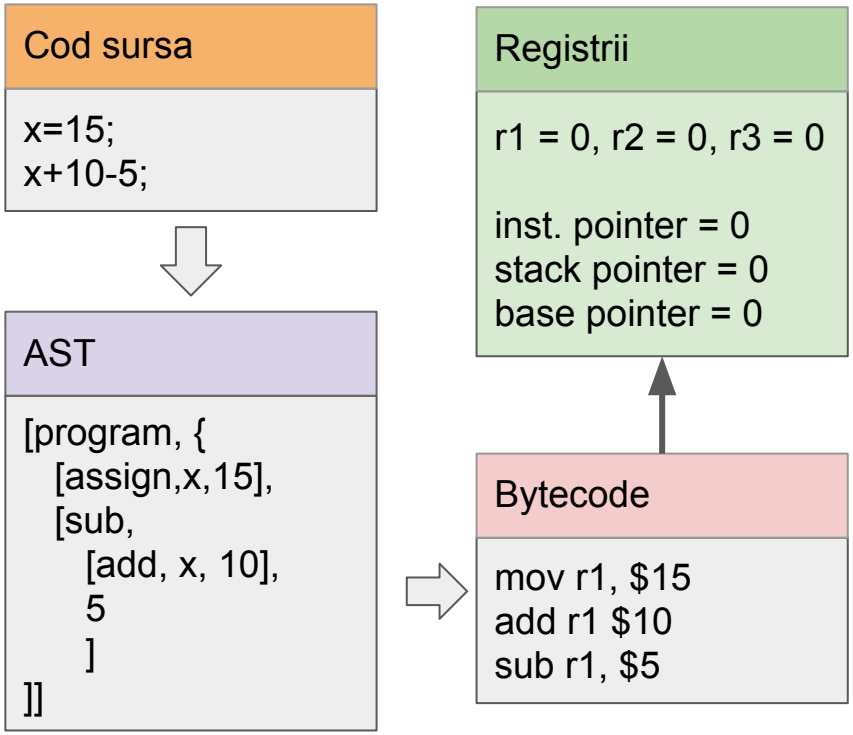


Tip Registru (Register based)

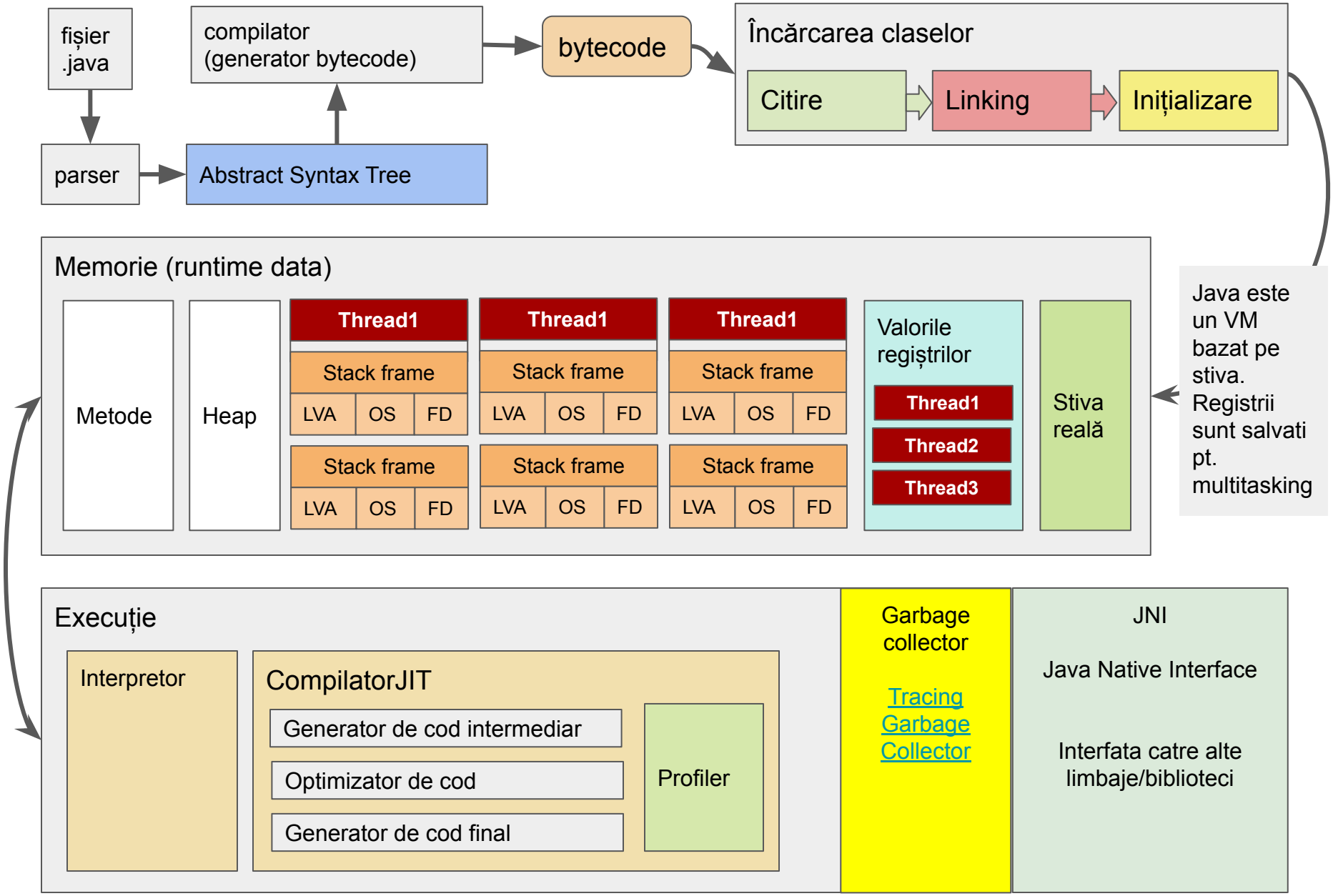
Lua, Parrot, Dalvik, Javascript

Mașinile virtuale de tip registru utilizează un set de registre virtuale (similare cu registrele procesorului).

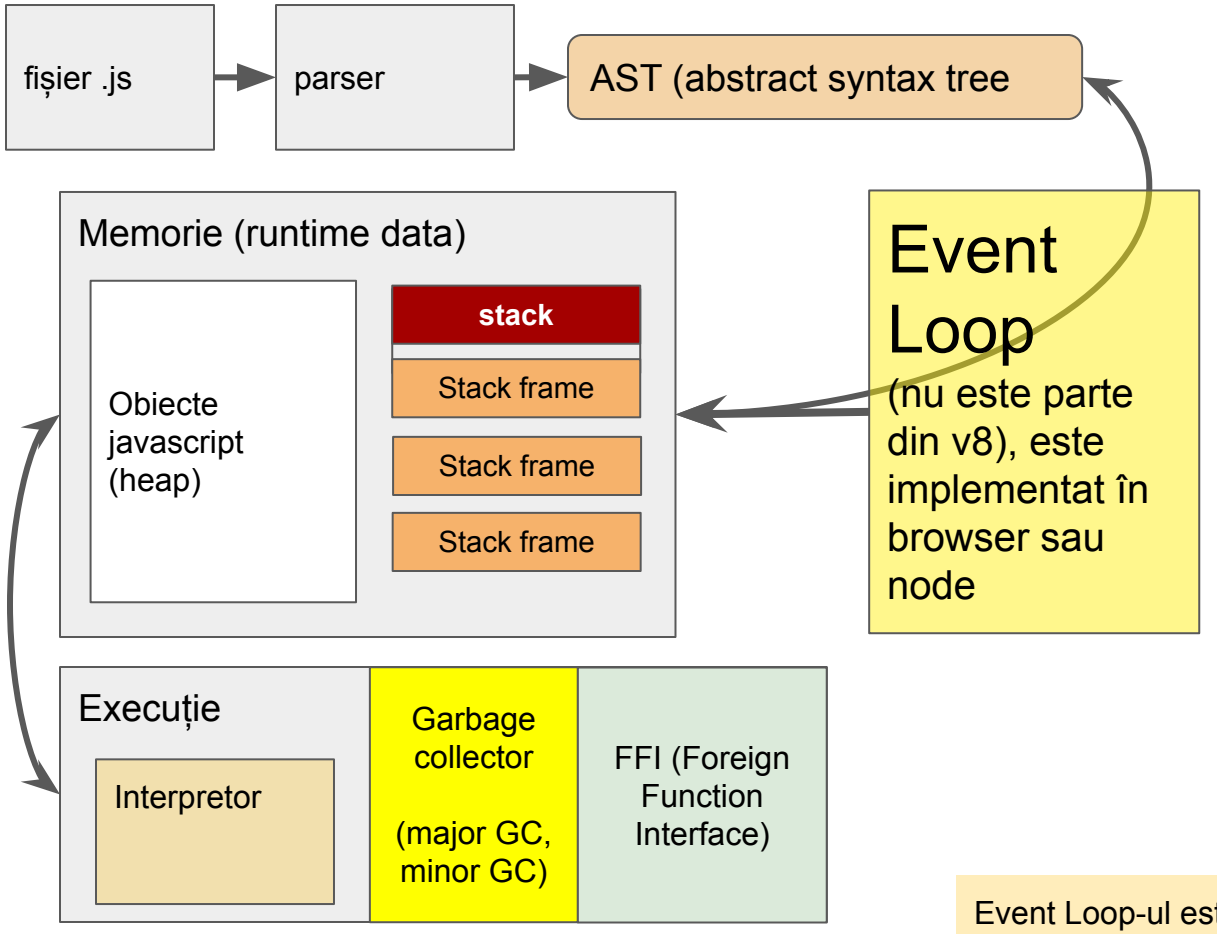
Rezultatul calculelor se găsește în registrul "acumulator"
Registrele VM se mapeaza registrelor reale ale procesorului.



Mașini virtuale de proces (JAVA)



Mașini virtuale de proces (Javascript V8)



Javascript V8 este un motor de execuție care are **un singur thread**.

Execuția javascript nu este niciodată directă, ea este mediată de browsere sau de componente suplimentare gen node.js. Acestea adaugă alte componente.

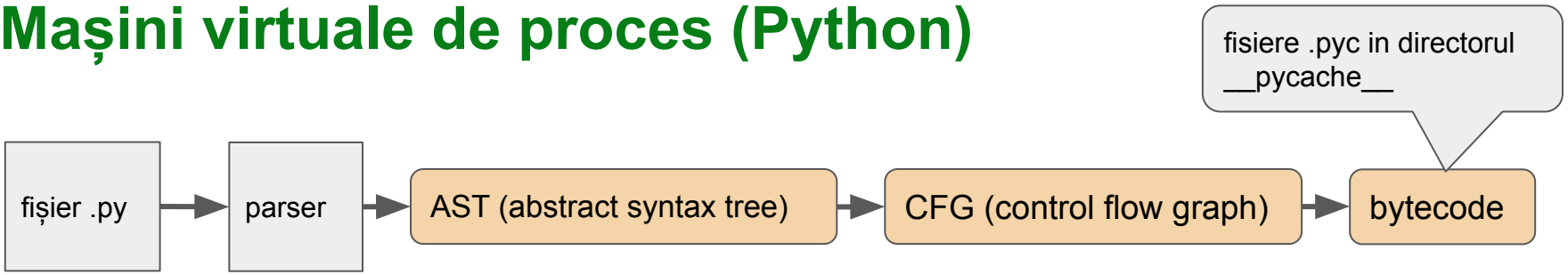
Aceste componente suplimentare permit lansarea concomitentă mai multor threaduri V8

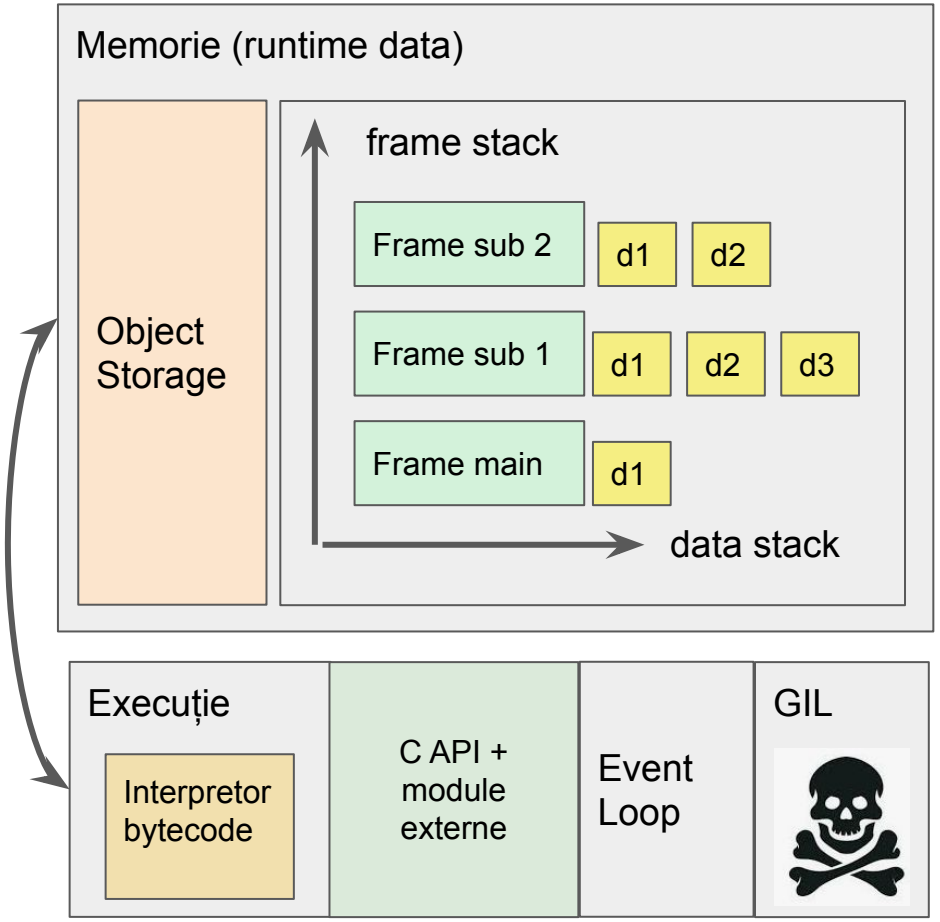
Aceste threaduri nu pot comunica între ele la fel de bine ca threadurile normale, de aceea este nevoie de **callback-uri** și de **event loop**

Event Loop se poate găsi și în Python, Php, Perl. Acesta este un component esențial al programării asincrone, orientate pe eveniment.

Event Loop-ul este un fel de scheduler de evenimente care execută operații suplimentare pe stack-ul v8.
Event Loop-ul este parte a browserului sau a node.js.
Apelurile AJAX (de exemplu) sunt făcute de browser, nu de V8.
Click-urile in browser sunt tot evenimente în Event Loop și au nevoie de callback-uri pentru a fi lansate în execuție la momentul potrivit.

Mașini virtuale de proces (Python)





Python nu are un compilator JIT inclus - exista module externe care permit aceasta numba, pyjon, care (cand functioneaza) pot aduce o creștere semnificativă de performanta

GIL (Global Interpreter Lock) [1992] reprezinta un mecanism de semaforizare de tip mutex care evita coliziunile mai multor fire de executie pe aceleasi variabile.

Semaforizarea este făcută la nivelul întregului thread - cu consecința că Python nu poate executa doua threaduri în același timp, dacă are mai multe procesoare/nuclee disponibile. La orice moment, doar threadul care detine GIL (Lockul) poate face modificări asupra structurilor din memorie.

CPython folosește reference counting pentru eliberarea majorității obiectelor (ca si php) și are un garbage collector suplimentar doar pentru detectarea referintelor ciclice

Extinderi ale paradigmei procedurale

Programele sunt împărțite în subprograme (funcții). Un subprogram, după apelare, rulează neîntrerupt până la return. Subprogramele nu pot fi executate decât în momentul apelarii lor. Subprogramele nu pot fi întrerupte și reluate.

- Extinderile paradigmei procedurale sunt elemente care depășesc modelul execuției bazate pe existența unei singure stive în care funcțiile (procedurile) sunt reprezentate în timpul execuției prin frame-uri.
- Aproape toate aceste extinderi sunt inspirate din multitasking-ul implementat la nivelul sistemelor de operare. Aproape toate sunt implementate prin extinderea modului de interacțiune cu stiva principală de execuție.
- Datorită necesității manipulării suplimentare a stivei de execuție, acestea pot fi implementate în limbajele indirect executate (interpretate). Apariția lor in limbaje cum este C++ se face prin mecanisme indrecte.
- Limbajul C nu suportă în mod direct aceste extensii datorită relației foarte directe pe care o are cu hardware-ul care oferă doar modelul de execuție bazat pe o singură stivă cu intervenții foarte clare.
- Dar, aproape că nu există program modern care să nu utilizeze frecvent aceste extensii. De aceea, este important să clarificăm modul de implementare a acestora.

Extinderi ale paradigmei procedurale

- Toate conceptele prezentate în cele ce urmează sunt în realitate concepte de programare, despre care se știe ce se așteaptă de la ele și cum trebuie să funcționeze. Detaliile de implementare sunt de obicei mai puțin cunoscute și ele pot varia semnificativ de la un limbaj la altul.

- Dar, acestea sunt concepte de programare care NU POT Fi ÎNȚELESE doar pe baza modelului procedural clasic, bazat pe o singură stivă de execuție în care se construiesc frame-uri pentru fiecare funcție apelată.

- De aceea acestea sunt chiar extensii ale paradigmei - ale principiilor fundamentale ale modului procedural. Chiar dacă unele dintre ele au fost inventate în programarea funcțională (funcțiile anonime, callbackurile) este important faptul că acestea pot fi folosite acum într-un cod procedural.

- Ele se transferă și în paradigma obiectuală - toate conceptele prezentate în continuare pot fi utilizate și în programarea orientată obiect, cu modificările de sintaxă și implementare necesare.

  - Callbacks
  - Funcții anonime (sau lambda)
  - Închideri (Closures)
  - Generatori
  - Corutine
  - Asincronicitate

Callback

Un callback este o funcție care poate fi trimisă ca argument unei alte funcții.

callback.php

```php
<?php

doIt('myCallback');

function doIt($callback)
{
  $sir = "I find your lack of faith disturbing.";
  $callback($sir);
}

function myCallback($argument)
{
  print 'Am primit: ' .  $argument .  "\n";
}
```

https://onlinegdb.com/HC1SZsKgf

Funcția “doIt” primește ca argument numele unei alte funcții (myCallback). În interiorul funcției, variabila $callback poate fi executată cu transmiterea argumentului $sir.

Limbajul C suportă callbackuri cu ajutorul pointerilor către funcții, ajutat de faptul că sistemul de programare procedurală oricum stochează pointeri către adresele de intrare ale funcțiilor.

Funcția qsort, aflată în biblioteca stdlib.h necesită ca argument o funcție de comparare a două elemente.

callback.c

```c
#include<stdio.h>

void callback(int n)
{
  printf("Eu sunt functia callback si am primit %d\n", n);
}

void executor(void (*ptr)(int))
{
  int mi = 13;
  (*ptr) (mi); // callback
}
int main()
{
  //se atribuie unui pointer o referinta catre functia callback
  void (*my_callback)() = &callback;
  //se lanseaza in executie functia executor cu argumentul ptr
  executor(my_callback);
  //executor(callback)  - si aceasta modalitate de apelare
functioneaza
  return 0;
}
```

https://onlinegdb.com/U6IPnLpVo

Funcții anonime

Funcții (sau proceduri) care pot fi definite fără identificatori (fără nume specific de funcție).

```php
<?php
$prn = function ($string)
{
   print "Sirul meu este: ". $string ."\n";
};

$prn("I find your lack of fate disturbing");
$prn("There's always a bigger fish");
```

https://onlinegdb.com/mEdU2bpkP

Funcția care determină afișarea șirurilor nu are nici un nume (este anonimă) dar poate fi asociată unei variabile și apelată ca atare.

```javascript
console.log('Incepem...')
setTimeout(function () {
   console.log('Executam dupa 1 secunda')
}, 1000);
```

Javascript utilizează foarte frecvent funcții anonime, existând funcționalități care nu pot fi atinse altfel.

Funcțiile anonime sunt folosite frecvent pentru aplicarea unor prelucrări tuturor elementelor unui tablou.

```php
<?php
$nume = ["computer", "kompjuter",
"ordinador", "urdinatore", "tietokone"];

$entities = array_map(fn($nm) => print
$nm."\n", $nume);
```

```javascript
var nume = ["computer", "kompjuter",
"ordinador", "urdinatore", "tietokone"];

nume.forEach(function(nm, index) {
 console.log(index + " " + nm);
});
```

Limbajul de programare C nu permite funcții anonime. Anumite compilatoare adaugă extensii care permit aproximatii ale funcțiilor anonime (gcc si clang de exemplu).

Închidere (Closure)

Funcții (proceduri) care pot fi definite în alte funcții și returnate, cu păstrarea parțială sau totală a frame-ului stivei.

Dacă putem asocia o funcție anonimă unei variabile înseamnă că putem să și returnăm o funcție din altă funcție?

closure.js

```
function topfunc() {
   var me = 'Sorin';
   var middlefunc = function(variabila) {
    print (variabila); //print pentru rhino
    print (me);  //print pentru rhino
   }
   return middlefunc;
}

var mf = topfunc();
mf('ionel');
```

https://onlinegdb.com/lGv0jGILV

```
Ionel
Sorin
```

Variabila “me” este definită în interiorul funcției “topfunc”. Ea nu ar trebui să fie disponibilă în afara acestei funcții.

Cu toate acestea, la execuția funcției mf() care este returnată din funcția topfunc, variabila “me” pare a fi disponibilă pentru a fi printată. DE CE??

O funcție definită astfel, în limbajul javascript, se transformă în ceea ce se numește “closure” sau “Închidere”.

O ”Închidere” este o funcție anonimă care înregistrează și “contextul” acesteia - adică variabilele funcției în care ea a fost definită.

Astfel, funcția anonimă care este definită în funcția **middlefunc** va purta după ea și variabilele definite în funcția “middlefunc” (În acest caz, variabila “me”) și le va trata ca și cum ar fi fost definite în interiorul ei.

Închidere (Closure)

```php
<?php

function topfunc() {
  $me = 'Sorin';
  $middlefunc = function($var) {
     print ($var."\n");
     print ($me."\n");
  };
  return $middlefunc;
}

$mf = topfunc();
$mf('Ionel');
```

closure-error.php

https://onlinegdb.com/1Q-dNebNKL

```
Ionel
PHP Notice:  Undefined
variable: me in
/home/main.php on line 7
```

```php
<?php

function topfunc() {
  $me = 'Sorin';
  $middlefunc = function($var)  use ($me) {
     print ($var."\n");
     print ($me."\n");
  };
  return $middlefunc;
}

$mf = topfunc();
$mf('Ionel');
```

closure-corect.php

https://onlinegdb.com/rThTGWj8N

```
Ionel
Sorin
```

php cere specificarea variabilelor care să însoțească funcția de tip closure, spre deosebire de javascript care le păstrează pe toate automat.

Închidere (Closure)

Cum sunt implementate închiderile?

1. Compilatorul crează o structură de date în memorie care va conține o referință către codul executabil al funcției, împreună cu variabilele care trebuie stocate și valorile lor.
2. La execuția funcției, variabilele stocate în structura respectivă de date sunt copiate în stack-ul funcției ca variabile definite local.

Structura de date nu poate fi stocată în stack pentru ca frame-ul funcției părinte se va șterge după încheierea execuției acesteia și se vor pierde variabilei definite.

De aceea, această structură de date se definește în heap. Dar heap-ul nu are nici un mecanism de eliberare automată a memoriei în momentul în care alocarea ei nu mai este necesară. Ce se întâmplă cu aceste structuri de date după ce nu mai sunt necesare?

De obicei, limbajele care fac astfel de manipulări suplimentare a conținutului stackului trebuie să implementeze o posibilitate de a elibera automat acea memorie. Acest mecanism se numește “gunoier” (garbage collector).

Limbajul de programare C nu permite închideri. Gcc adaugă o extensie care permite definirea a ceea ce se numește “nested functions” . Acestea nu pot fi returnate, viața lor durează exclusiv în interiorul funcției părinte.

Generatori

Generatorii sunt funcții care pot ceda voluntar, temporar controlul funcției apelante.

```php
<?php

function Generator()
{
  echo "computer","\n";
  yield;
  echo "kompjuter","\n";
  yield;
  echo "ordinador","\n";
  yield;
  echo "urdinatore","\n";
  yield;
}

$gen = Generator();

$value = $gen->current();
$gen->next();
$gen->next();
```

https://onlinegdb.com/2gyA0B6O-

```
php generator.php

computer
kompjuter
ordinador
```

```php
<?php

function Generator()
{
  yield "computer";
  echo "dupa primul yield\n";
  yield "kompjuter";
  echo "dupa al doilea yield\n";
  yield "ordinador";
  echo "dupa al treilea yield\n";
  yield "urdinatore";
  echo "dupa al patrulea yield\n";
}

$gen = Generator();
$value = $gen->current();
print $value."\n";
$gen->next();
$value = $gen->current();
print $value."\n";
$gen->next();
$value = $gen->current();
print $value."\n";
```

https://onlinegdb.com/KM2ngk1cn

```
php generator1.php

computer
dupa primul yield
kompjuter
dupa al doilea yield
ordinador
```

Generatorii pot fi și funcții anonime

```php
<?php

$mygen = function()
{
  yield "computer";
  echo "dupa primul yield\n";
  yield "kompjuter";
  echo "dupa al doilea yield\n";
  yield "ordinador";
  echo "dupa al treilea yield\n";
  yield "urdinatore";
  echo "dupa al patrulea yield\n";
};

$gen = $mygen();
$value = $gen->current();
print $value."\n";
$gen->next();
$value = $gen->current();
print $value."\n";
$gen->next();
$value = $gen->current();
print $value."\n";
```

https://onlinegdb.com/lehlxOvIj

```
php generator1.php

computer
dupa primul yield
kompjuter
dupa al doilea yield
ordinador
```

Generatori

generator1.php

```php
<?php

function Generator()
{
   yield "computer";
   echo "dupa primul yield\n";
   yield "kompjuter";
   echo "dupa al doilea yield\n";
   yield "ordinador";
   echo "dupa al treilea yield\n";
   yield "urdinatore";
   echo "dupa al patrulea yield\n";
}

$gen = Generator();

$value = $gen->current();
print $value."\n";
$gen->next();
$value = $gen->current();
print $value."\n";
$gen->next();
$value = $gen->current();
print $value."\n";
```

Funcția Generator este apelată și începe să se execute. Execuția ei se suspendă (dar fără a se șterge stack frame-ul și fără returnare) la întâlnirea primei instrucțiuni yield. Yield poate returna sau nu o valoare.

La apelarea metodei “next”, execuția funcției Generator se reia din punctul în care a rămas, până la următorul yield. Toate informațiile asociate funcției (variabile locale, etc.) sunt păstrate. Funcția se execută până la următorul yield.

Metoda “next” se poate apela de câte ori este nevoie, până la epuizarea tuturor yield-urilor.

https://onlinegdb.com/KM2ngk1cn

Limbajul C nu suporta direct generatori. Există o serie de extensii ale acestuia care pot adăuga această funcționalitate.

PseudoGeneratori

```php
<?php

$stare = 0;

function PseudoGenerator() {
   global $stare;
   if ($stare == 0) {
       $stare = 1;
       return "computer";
   } else if ($stare == 1) {
       $stare = 2;
       return "kompjuter";
   } else if ($stare == 2) {
       $stare = 3;
       return "urdinatore";
   } else if ($stare == 3) {
       $stare = 4;
       return "ordinador";
   }
}

$val = PseudoGenerator();
print "Val: ". $val ."\n";

$val = PseudoGenerator();
print "Val: ". $val ."\n";

$val = PseudoGenerator();
print "Val: ". $val ."\n";
```

https://onlinegdb.com/QZLPJEBuq

Generatori fără yield

Cheia implementării unui generator este menținerea informației de stare.

Aceasta se poate emula folosind o variabilă globală care poate fi accesibilă din interiorul funcției.

Un apel succesiv al funcției va returna valori diferite. Diferența este că funcția se va executa de fiecare dată de la început până la return-ul corespunzător.

În implementările reale, întreaga stivă a funcției este înregistrată în heap (la apelarea instrucțiunii yield) și este reîncărcată în stiva principală la apelarea metodelor de tip next. Astfel, rămân disponibile la reluarea execuției toate variabilele locale definite în funcție.

Acest lucru este posibil în limbajele interpretate, executate în mașini virtuale pentru că acestea pot manipula stackul fără constrângerile impuse de hardware.

PseudoGeneratori în C

```c
#include <stdio.h>

int numara(void) {
   static int i, stare = 0;
   switch (stare) {
       case 0:
       for (i = 0; i < 10; i++) {
           stare = 1;
           return i;
           case 1:;
       }
   }
}

int main() {
   int c;
   c = numara();
   printf("c este: %d\n", c);
   c = numara();
   printf("c este: %d\n", c);
   c = numara();
   printf("c este: %d\n", c);
}
```

Variabilele care au clasa de stocare “static” sunt stocate în segmentul de date al memoriei și sunt persistente pe toată durata execuției - disponibile doar în funcția curentă. Astfel, pot fi utilizate pentru menținerea stării între apeluri succesive ale funcției.

Costrucția switch - case poate fi intercalată printre alte instrucțiuni astfel încât este posibil ca un case să fie în interiorul unei bucle for în timp ce switchul corespunzător se află în afara acestuia.

Atribuirea valorii 1 variabilei “stare” face ca la următoarea execuție a funcției construcția “case” să determine saltul direct în interiorul ciclului, după instrucțiunea “return”. Ciclul va sări la următorul i pe care îl va returna.

https://onlinegdb.com/xUIMXBk7U

Această utilizare a construcției “case” a fost probabil propusă de un angajat al companiei Lucasfilm numit “Tom Duff” care a folosit-o pentru optimizarea unei bucle de copiere. Optimizarea buclei este cunoscută sub numele “Duff’s device” (mai multe detalii: https://en.wikipedia.org/wiki/Duff%27s_device)

Corutine

Corutinele sunt asemănătoare subrutinelor, cu deosebirea că execuția lor poate fi suspendată și reluată de mai multe ori. Informațiile de stare ale unei corutine se mențin între rulări succesive.

Spre deosebire de generatori, corutinele pot primi parametri la rulările succesive.

Corutinele mențin propriul stack frame, în diverse moduri, în funcție de implementare. Astfel, corutinele sunt similare firelor de execuție (green threads) cu deosebirea că acestea cedează controlul voluntar (multitasking cooperativ), in timp ce firele de execuție sunt de cele mai multe ori întrerupte de planificator (multitasking preemptiv).

```
def filter_line(ac):
    while True:
        line = (yield)
        if ac in line:
            print(line)


cor = filter_line("om")
next(cor)
cor.send("tietokone")
cor.send("computer")
cor.send("ordinateur")
cor.send("számítógép")
cor.send("ríomhaire")
```

https://onlinegdb.com/GGOaydknw

```
python coro1.py

computer
ríomhaire
```

Se definește funcția filter_line care este de tip corutina și care primește argumentul “ac”.

while True asigură execuția liniilor următoare de un număr infinit de ori.

Dacă este găsit șirul din variabila ac în interiorul șirului din variabila line, acesta este imprimat la consola.

Funcția filter_line este apelată, spre deosebire de apelarea unei funcții normale, aceasta NU este lansată în execuție.

Instrucțiunea next(cor) lansează funcția în execuție. Aceasta va fi executată până la prima instrucțiune (yield), unde este suspendată și începe să aștepte primirea unui argument

Se trimite parametrul “tietokone” care nu conține șirul “om”. Corutina va executa instrucțiunile până la următorul yield, unde se va opri si va aștepta.

Se trimite parametrul “computer” care nu conține șirul “om”. Corutina va executa instrucțiunile până la următorul yield, afișând la consolă șirul primit pentru că acesta conține “om”, apoi se va opri și va aștepta.

Corutine

```
def filter_line(num):
   while True:
       line = (yield)
       if num in line:
           yield(line)

cor = filter_line("om")
next(cor)
a = cor.send("tietokone")
if a: print(a)
a = cor.send("computer")
if a: print(a)
a = cor.send("ordinateur")
if a: print(a)
a = cor.send("számítógép")
if a: print(a)
a = cor.send("ríomhaire")
if a: print(a)
```

coro2.py

```
python coro2.py

computer
ríomhaire
```

https://onlinegdb.com/ryxHMkZw8u

Corutina poate returna parametrii, la fel ca generatorul. Aceasta se face de obicei tot cu ajutorul instrucțiunii yield. Compilatorul va detecta tipul de operație în funcție de sintaxă.

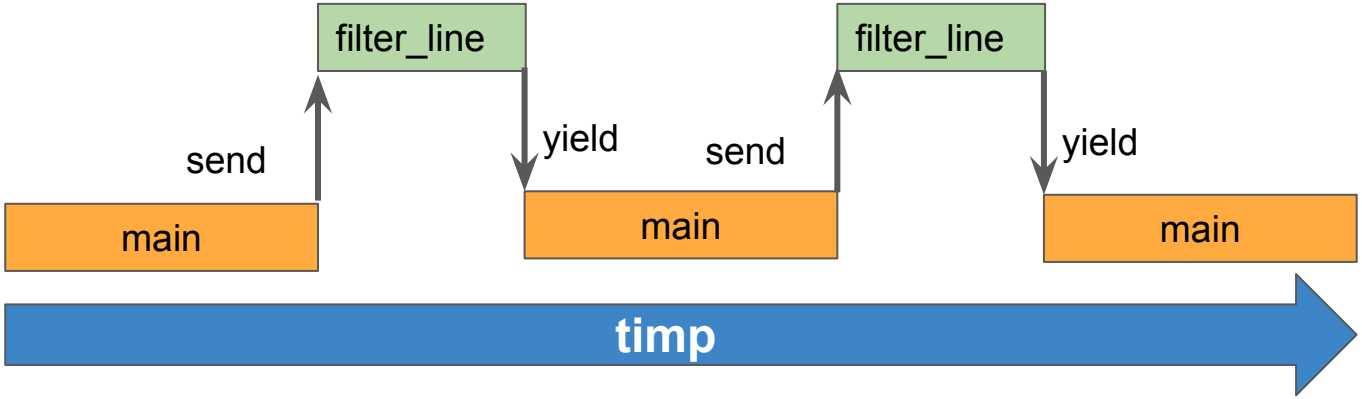
Limbajul C NU suportă corutine nativ datorită relației directe cu hardware-ul. Biblioteca standard C oferă un mecanism de salt necondiționat către o altă funcție, fără distrugerea stivei curente, folosind funcțiile setjmp și longjmp. Pe baza acestui mecanism, împreună cu funcțiile setcontext și getcontext, tot din biblioteca standard, au fost implementate mai multe tipuri de corutine.

Exemple:

https://code.google.com/archive/p/libconcurrency/
http://software.schmorp.de/pkg/libcoro.html
https://github.com/aolarchive/ribs2

Corutine - fire de execuție

Cele două funcții (main, funcția principală și filter_line, corutina apelată) nu rulează niciodată în același timp. Dar, ele există în același timp în memorie, fiecare dintre ele cu propriul frame de stivă.



Funcția “filter_line” cedează de fiecare dată voluntar controlul, în acest caz așteptând următorul parametru de la funcția apelantă. Cele două funcții se află într-o relație de multitasking cooperativ.

Corutina este un concept teoretic - acesta descrie un anumit comportament, nu o anumită implementare. De aceea, detaliile de implementare pot diferi semnificativ de la un limbaj de programare/compilator la altul. Anumite compilatoare au segmente speciale pentru implementarea corutinelor, altele se bazează pe threaduri sau fibre (în special fibre dacă sunt disponibile datorită multitaskingului cooperativ necesar).

Mecanismele descrise până acum se execută pe un singur procesor (nucleu)! Acestea reprezintă un mod de programare concurentă dar NU reprezintă în nici un fel paralelism.

Async - Await

Cu ajutorul multitaskingului cooperativ oferit de corutine și generatori, împreună cu closures și funcții anonime, este posibilă **o mai bună optimizare a proceselor de prelucrare a datelor** prin evitarea operațiilor care “blochează” procesorul degeaba. Majoritatea programelor de consum de date moderne folosesc asincronicitatea pentru a scădea timpul total de execuție, chiar în situația în care taskurile sunt executate pe același procesor.

Orice pagină WEB care solicită resurse adiționale textului propriu-zis (imagini, fișiere css, js, informații din alte surse, etc.) nu ar afișa absolut nimic tip de câteva secunde dacă nu ar permite afișarea asincronă a textului în timp ce se așteaptă încărcarea resurselor adiționale.

Din ce în ce mai multe limbaje de programare implementează **mecanisme de programare asincronă**. Cele mai frecvent utilizate cuvinte cheie în acest domeniu îl reprezintă perechea “**async/await**”.

De ce este importantă asincronicitatea?

Aproape toate programele care rulează pe un computer modern prelucrează cantități mari de date pe care trebuie să le obțină de la surse foarte lente. Harddiskul este o sursă lentă dar arhitecturile moderne implică obținerea datelor nu numai de la harddiskuri ci și de la surse externe aflate undeva pe internet.

O funcție care așteaptă date poate ceda controlul unei altei funcții urmând să-l primească înapoi atunci când începe să primească date. În acest mod se elimină timpii de așteptare.

În mod normal această treabă ar trebui făcută de către sistemul de operare dar pentru aceasta este nevoie de programare multiproces/multithread. Este dificilă, consumă foarte multe resurse (inițializarea proceselor durează). Astfel, se implementează o versiune de asincronicitate în interiorul unui singur program.

Async - Await

async: Este un cuvânt cheie care marchează o funcție ca fiind asincronă. O funcție marcată cu async returnează un obiect care poate fi **awaited** (de obicei, un Promise sau un tip similar).

await: Permite așteptarea unui obiect asincron până când acesta este complet, fără a bloca întreaga aplicație.

În funcție de limbajul de programare, **async** va returna fie o **promisiune** (în limbaje cu execuție imediată precum JavaScript sau C#), fie o **corutină** (în limbaje cu execuție întârziată precum Python sau Rust). **Promisiunile** pot fi văzute ca o **versiune automată** a unui **callback**, dar cu un mecanism mai structurat și mai ușor de gestionat.

Promisiuni	Corutine
Sunt obiecte care reprezintă un rezultat care va fi disponibil în viitor.	Sunt funcții care pot fi suspendate și reluate, permițând execuția asincronă.
pot fi înlănțuite folosing .then() construind astfel un fir asincron de execuție	await-uri succesive își păstrează ordinea de execuție.
Nu pot fi suspendate; sunt întotdeauna "în așteptare" până când se rezolvă.	Pot fi suspendate și reluate oricând în timpul execuției.
Javascript, C#, Swift	Python, Kotlin, Rust, Lua

Async - Await - analogie

O maestră internațională la șah, foarte drăguță și extrem de bine îmbrăcată, își propune să joace un simultan cu 24 de jucători amatori.

Premise

- 24 de amatori
- Maestra are nevoie de 5 secunde pentru o mutare
- Fiecare amator are nevoie de 55 de secunde pentru o mutare
- Mutarile sunt făcute de o singura persoană (nici maestra nici amatorii nu ating tablele)
- Persoana care face mutarile se poate muta între table foarte repede (instantaneu)
- Jocurile au în medie 30 de perechi de mutări.

Varianta sincronă

Maestra joacă fiecare partidă separat, una câte una, fără să înceapă următoarea partidă până când partida curentă nu se termină.

Fiecare partidă durează 30 de minute *(55 + 5) * 30 = 1800 secunde*.

Întregul simultan durează 12 ore.

Varianta asincronă

Maestra se mută de la o tablă la alta, făcându-și mutarea lăsându-i pe amatori să se gândească tip de 55 de secunde.

O mutare a maestrei pentru toate cele 24 de jocuri durează 2 minute (*24 * 5 == 120* seconds), fiecare amator are la dispoziție cele 55 de secunde pentru a-și face mutarea.

Întregul simultan durează doar 1 oră (*120 * 30 == 3600* seconds).

Multitaskingul cooperativ NU paralelizează execuțiile, paralelizează asteptările.

Async - Await

https://onlinegdb.com/okFQpCjzO

```
import time

def show():
   print("kompjuter")
   time.sleep(1)
   print("ordinador")
   time.sleep(1)
   print("computer")

def main():
    show()
    show()
    show()
main()
```

```
time python sync-sleep.py
kompjuter
ordinador
computer
kompjuter
ordinador
computer
kompjuter
ordinador
computer

real    0m6,040s
user    0m0,031s
sys     0m0,003s
```

https://onlinegdb.com/hlcIw12Wy

```
import asyncio

async def show():
   print("kompjuter")
   await asyncio.sleep(1)
   print("ordinador")
   await asyncio.sleep(1)
   print("computer")

async def main():
   await asyncio.gather(show(),
show(), show())

asyncio.run(main())
```

```
time python async-sleep.py
kompjuter
kompjuter
kompjuter
ordinador
ordinador
ordinador
computer
computer
computer

real    0m2,101s
user    0m0,088s
sys     0m0,008s
```

Se definește funcția asincronă “show”. De fapt, async transformă funcția show() într-o corutină.

await suspendă execuția corutinei si transfera controlul către funcția care a apelat-o.

Funcțiile asincrone trebuie apelate tot cu ajutorul funcției await.

Limbajul de programare C nu suportă programare asincronă. Există implementări care utilizează bibliotecile multithread ale sistemului de operare si exista posibilitatea asincronicitatii in I/O.

https://www.programiz.com/python-programming/online-compiler/

Async - Await

Tranziția de la programarea secvențială la programarea asincronă (concurentă) este probabil cea mai importantă modificare de paradigmă care s-a întâmplat în lumea programării în ultimii 10 ani.

Tranziția a fost determinată mai degrabă de nevoie decât de inovație - modelele concurente de execuție existau cu mult timp înainte de a apărea această creștere masivă a utilizării lor.

Această tranziție a făcut posibilă apariția unor noi arhitecturi software (ex: microservicii), noi produse (interfețele Web complexe) și a contribuit la creșterea vitezei majorității instrumentelor software cu care interacționăm frecvent.

Modul în care sunt implementate elementele de programare asincronă diferă de la o implementare a unui limbaj de programare la altul, multe limbaje având chiar mai multe modele. Acest mode de implementare se mai numește și "**modelul concurențial**" sau "**modelul de concurență**"

Toate implementările au în comun faptul că au nevoie de un component separat pentru managementul taskurilor asincrone. Acesta poate fi un event loop, un grup de task-uri sau threaduri, cu sau fără ajutorul sistemului de operare. Dar, de fiecare dată este nevoie de un instrument suplimentar pentru menținerea unui registru al activităților asincrone.

De exemplu, motorul de execuție Javascript folosit in Google Chrome (numit V8) nu permite asincronicitate. Aceasta este adăugată ca parte a browserului. Node.js, care este un motor de execuție a programelor javascript pe server, deși folosește tot v8 are un alt component care asigură asincronicitatea. Chrome folosește propriile mecanisme de implementare a event loop-ului iar node.js folosește https://libuv.org/. Ambele se folosesc de apeluri de sistem al sistemului de operare.

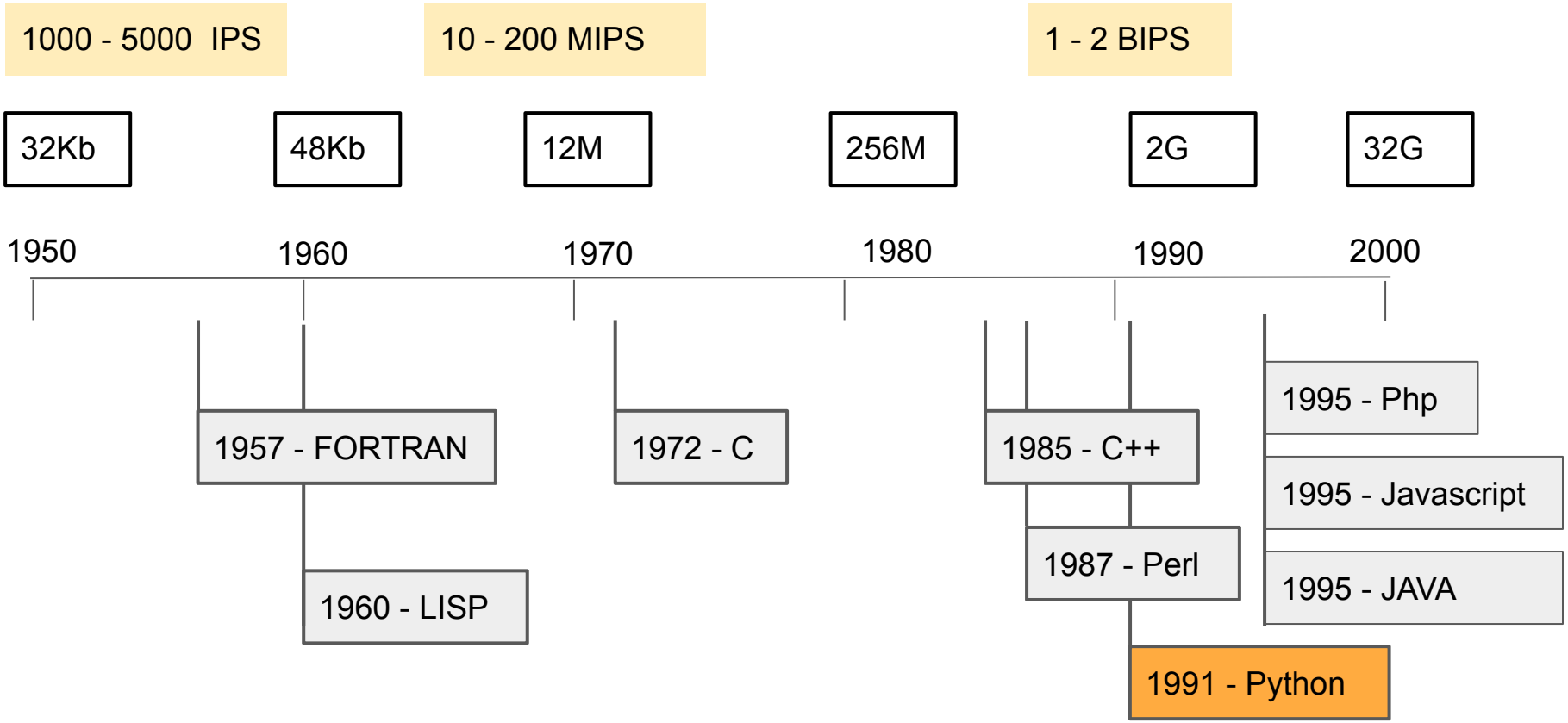
v8 nu contine instrucțiunea setTimeout()!! Ea, împreună cu altele similare sunt implementate în event loop.

python

python™

47 Momentul apariției Python

Pana in anii 1980 toate limbajele de programare erau concepute pentru maximizarea hardware-ului.



Python a "învățat" din majoritatea limbajelor preexistente, în special C, Perl.

Procesul apariției Python

48

Începerea implementarii	Python 0.90	Python 1.0	Python 2.0	Python 3.0	Python 3.6
1989	1991	1994	2000	2008	2016

Denumirea "Python" este inspirată de trupa de comedie britanica Monty Python

Guido van Rossum

- Nascut in 1956, în Olanda
- A absolvit Universitatea din Amsterdam, in 1982
- Medalie de bronz la olimpiada internationala de matematica, in 1974

CWI (Olanda) | NIST (USA) | CNRI (USA)

Be.open (USA) | Zope (USA) | Elemental Security (USA)

Google (USA) | Dropbox (USA) | Microsoft (USA)

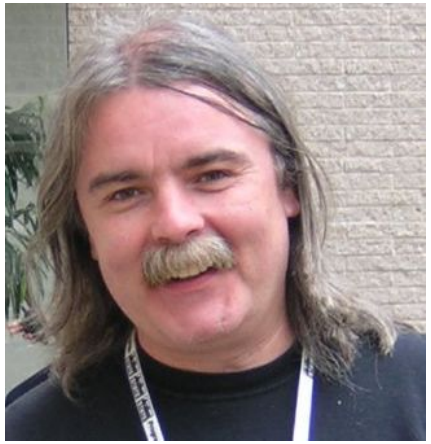
Python

- Un limbaj de programare cu multe domenii de utilizare
    - Medii de construcție WEB ca Django, Flask, Bottle Cherrypy, Dash, Falcon)
    - Programare GUI (Tkinter PyQT, wxPython, PyGTK)
    - Scripturi și automatizări de sistem (Ansible, BUildBot, Fabric)
    - Programare stiintifica (Inteligenta artificiala, calcul matriceal, prelucrări de imagini, etc)
    - Programe de prelucrare a textului (inclusiv NLP)
- Un limbaj de programare implementat direct orientat obiect (dar care permite o programare multi-paradigma)
- Un limbaj cu tipizare dinamica dar puternica
- Un limbaj interpretat
- Un limbaj conceput pentru productivitatea programării
- Un limbaj cross-platform - codul python poate fi rulat pe aproape orice sistem de operare
- Biblioteca standard este foarte vastă iar bibliotecile disponibile sunt extrem de multe
- Un limbaj care are un ecosistem extrem de bine condus - management centralizat de biblioteci, multiple sisteme de construcție a mediilor virtuale, etc.

Cine folosește Python ?

Google
Instagram
airbnb
Goldman Sachs
yahoo!
NOKIA
Dropbox
Pinterest
moz://a
NIST
National Institute of Standards and Technology
Prezi
twitter
NETFLIX
PIXAR
JPMorganChase
lyft
Securities and Exchange Commission
hp
Microsoft
Spotify
intel
IBM
twilio
UBS
facebook
NASA
INDUSTRIAL LIGHT & MAGIC
Central Intelligence Agency
GETCO
amazon
Los Alamos NATIONAL LABORATORY
CITADEL
PayPal
Uber
EVE ONLINE
CISCO
Fermilab
slack
Quora
reddit
Qualcomm
Seagate
the ONION America's Finest News Source
ebay

The zen of Python - Tim Peters (2004)



- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Complex is better than complicated.
- Flat is better than nested.
- Sparse is better than dense.
- Readability counts.
- Special cases aren't special enough to break the rules.
- Although practicality beats purity.
- Errors should never pass silently.
- Unless explicitly silenced.
- In the face of ambiguity, refuse the temptation to guess.
- There should be one-- and preferably only one --obvious way to do it.
- Although that way may not be obvious at first unless you're Dutch.
- Now is better than never.
- Although never is often better than *right* now.
- If the implementation is hard to explain, it's a bad idea.
- If the implementation is easy to explain, it may be a good idea.
- Namespaces are one honking great idea -- let's do more of those!

https://onlinegdb.com/xRki8Ai3s

The zen of Gâgă (cca 1960, România)

Înțelepciune…

Decât urât, bolnav, bătrân și sărac, mai bine frumos, sănătos, tânăr și bogat

In cate feluri se poate face un calcul de frecventa?


```
item frequency = {}
  for item in items:
      if item not in item frequency:
          item frequency[item]  = 0
      item_frequency[item]  += 1
```

```
item frequency = defaultdict(int)
for item in items:
    item_frequency[item]  += 1
```

```
item_frequency = {item: items.count(item)  for item in items}
```

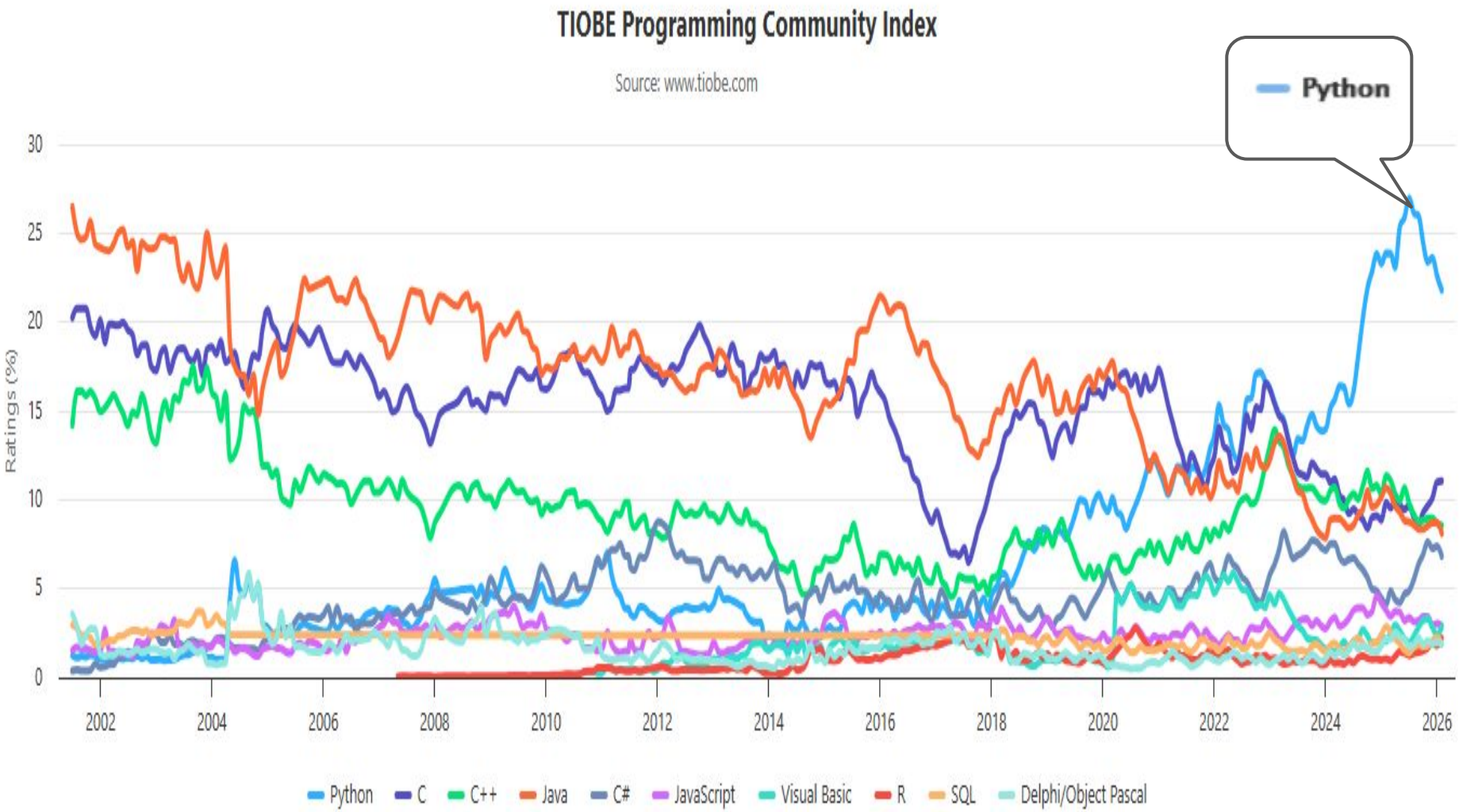
```
item_frequency = dict((item, items.count(item))  for item in items)
```

- There should be one-- and preferably only one --obvious way to do it.

Aforismul "**There should be one-- and preferably only one --obvious way to do it.**" era un atac direct la limbajul Perl al carui motto era "***There's more then wan way to do it***". Perl face aceasta afirmație ca pe o exprimare a posibilității de a utiliza stilul personal de programare, recunoscand și incurajand variabilitatea și diversitatea lumii. Python, ceva mai tanar și mai naiv, își dorește ordine, disciplină și uniformitate.

E tot timpul o idee bună ca orice declarații care conțin imperative ascunse ("should") făcute despre altceva decât propria persoană, în special despre grupuri întregi de oameni sau obiceiuri, să fie ignorate.

Popularitatea Python

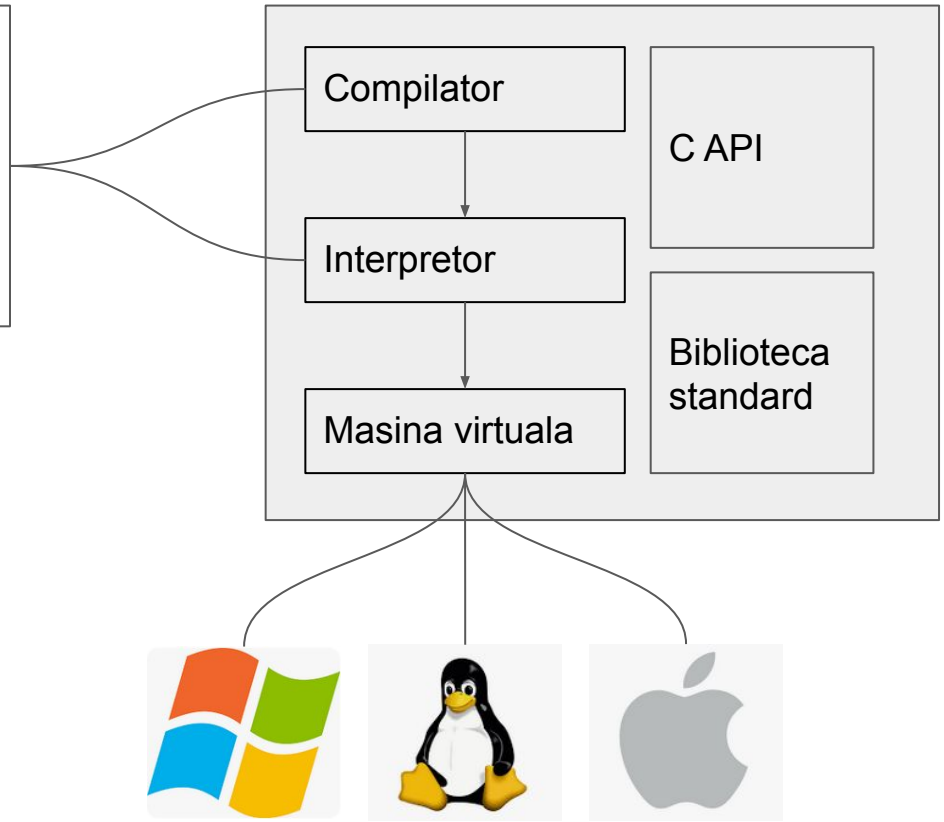


CPython - interpretorul de referința (oficial) al Python

Ce este Python?

Python este un limbaj de programare.

Python este un limbaj interpretat sau un limbaj compilat?



Da

Codul sursa Python este mai întâi compilat într-o secvența de bytecode. Apoi, aceasta este interpretata.

Variante CPython

- CrossTwine Linker - o combinație între CPython și o bibliotecă suplimentară care oferă performanțe îmbunătățite (în prezent proprietare)
- Stackless Python - varianta modificata de CPython cu accent pe concurență (utilizat de dspython pentru Nintendo DS) care decuplează interpretorul de stiva C
- Pyston - un fork al CPython, dezvoltat inițial la Dropbox (între timp independent), axat pe performanță. Adaugă accelerarea bytecode-ului și un JIT minimal.
- Cinder, un fork al CPython de la Instagram, care conține o serie de optimizări, cum ar fi rescrierea bytecode și un compilator JIT.
- Pyjion, o extensie JIT pentru CPython care compilează codul Python în CIL (.NET) și îl execută pe CLR (Common Language Routine, runtime-ul .NET).
- unladen-swallow - o varianta de optimizare a CPython, menită să fie pe deplin compatibilă și semnificativ mai rapidă, luată în vedere inițial pentru includere in CPython (PEP 3146). Autorii au renunțat la dezvoltarea acesteia.
- wpython - o reimplementare a CPython folosind „wordcode” în loc de bytecode (16 biți în loc de 8 biți pe instrucțiune)
- eGenix PyRun - Runtime Python (CPython + biblioteca standard) comprimate într-un singur binar de 3-4MB
- x-python - Interpretul C Python scris în Python; utile în scopuri educaționale. Există, de asemenea, depanator bytecode pentru acesta
- Falcon - o mașină virtuala bazata pe registrii disponibila ca modul

Alte implementari Python

- PyPy - Un alt compilator/interpretor Python care include un compilator JIT
- Jython - Interpreter Python scris in in Java
- IronPython - Impolementare Python in C# cu ajutorul Common Language Runtime (CLR/.NET)
- GraalPython - Python in Java, folosind compilatorul JIT Graal si cadrul de interpretare Truffle
- Skybison - O implementare C++ a Python 3.8 utilizând un Garbage Collector, clase ascunse si un JIT.
- Brython - Posibilitatea de a rula Python in browser prin traducerea in JavaScript
- CLPython - Python in Common Lisp
- HotPy - o masina virtuala alternativa care adaugă optimizări și traducere în cod nativ pe baza unul analizor de tip
- pyjs - (anterior numit Pyjamas) un compilator Python catre JavaScript cu un framework WEB adaugat
- PyMite - Python pentru embedded devices
- pyvm - a Python-related virtual machine and software suite providing a nearly self-contained "userspace" system
- RapydScript - un limbaj asemanator cu Python care se compilează in JavaScript
- tinypy - o implementare minimala a Python in 64k
- Transcrypt - Un alt compilator Python catre JavaScript rapid, cu typechecking optional, support JSX

Compilatoare și acceleratoare

- Cython - un compilator Python-to-C utilizat pe scară largă. Poate genera module de extensie CPython.
- Nuitka - un compilator Python->C++ care folosește libpython, încercând unele optimizări. Interacționează cu CPython.
- MyPyC compilează codul Python într-o extensie C, bazată pe mypy (care este un type checker pentru python transformându-l intr-un limbaj static tipizat).
- 2c-python - un compilator static Python->C, care traduce bytecodul CPython în C
- Compyler - o încercare de a „translitera bytecode-ul în limbaj de asamblare x86” (abandonat)
- GCC Python Front-End - un proiect care isi propune compilarea codului Python cu infrastructura GCC (Gnu C Compiler, principalul compilator C, C++, etc. din Linux).
- Pyc - analizor static cu scopul compilarii programelor Python, utilizează tehnici similare cu Shed Skin.
- Shed Skin - un compilator Python->C++, limitat la un subset implicit tipizat static al limbajului pentru care poate deduce automat tipuri eficiente prin analiza întregului program.
- unPython - un compilator Python -> C care utilizează adnotări pentru tipuri
- VOC - convertor al bytecode-ului Python in bytecode Java (transpiler).

Numerical Accelerators

- Numba - Compilator Python care poate fi folosit cu NumPy
- Pythran - compilator pentru un subset al Python cu accent pe calculul stiintific
- Copperhead -elemente de limbaj pentru calcule parallel in Pythoncare se poate compila pe CUDA sau alte backenduri multi-core.
- Parakeet - compilator pentru un subset numeric al Python

Cum arată Python?


```python
import random


    """
        Functie care sorteaza o lista
        prin metoda bubble sort
    """

    changed = True
    while changed:
        changed = False
        for i in range(len(seq) - 1):
            if seq[i] > seq[i+1]:
                seq[i], seq[i+1] = seq[i+1], seq[i]
                changed = True
    return seq


n = 10

#generam aleator o lista de n elemente

lista = [random.randrange(1, 100) for i in range(n)]

print(lista)
bubble_sort(lista)
print(lista)
```

https://onlinegdb.com/Dg7EkDlta

import al modulelor

functii ne-tipizate

comentariu pe mai multe linii (docstring)

boolean

generatoare de liste

blocurile de cod încep cu : si sunt obligatoriu indentate

variabilele nu sunt tipizate

comentariu pe o singura linie

tipuri de date multiple cu iteratoare incluse și acces la membrii (list comprehension)

call by object reference

Cum arata Python fata de C ?

```python
import random

def bubble_sort(seq):
    """
        Functie care sorteaza o lista
        prin metoda bubble sort
    """

    changed = True
    while changed:
        changed = False
        for i in range(len(seq) - 1):
            if seq[i] > seq[i+1]:
                seq[i], seq[i+1] = seq[i+1], seq[i]
                changed = True
    return seq


n = 10

#generam aleator o lista de n elemente

lista = [random.randrange(1, 100, n) for i in
range(n)]

print(lista)
bubble_sort(lista)
print(lista)
```

https://onlinegdb.com/Dg7EkDlta

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void bubble_sort (int *a, int n) {
    int i, t, j = n, s = 1;
    while (s) {
        s = 0;
        for (i = 1; i < j; i++) {
            if (a[i] < a[i - 1]) {
                t = a[i];
                a[i] = a[i - 1];
                a[i - 1] = t;
                s = 1;
            }
        }
        j--;
    }
}

int main () {
 int n = 10;
 int a[10];
 int i;

 srand(time(NULL));
 for (i = 0; i < n; i++) {
      a[i] = rand() % 50;
 }
 for (i = 0; i < n; i++) {
   printf("%d%s", a[i], i == n - 1 ? "\n" : " ");
 }
 bubble_sort(a, n);
 for (i = 0; i < n; i++) {
   printf("%d%s", a[i], i == n - 1 ? "\n" : " ");
 }
   return 0;
}
```

https://onlinegdb.com/vT6bL-5LV

Tipuri de date in Python

Toate tipurile de date in Python sunt clase iar datele propriu-zise sunt instanțe ale acestora.

Tip	Clasa	Descriere
Numeric	int, float, complex	Tipuri de date pentru valori numerice, singulare
String	str	Siruri de caractere
Secventa	list, tuple, range	Date cate stochează colecții de valori
Mapare	dict	Stocheaza perechi cheie-valoare
Boolean	bool	Tip de date True-False
Set	set, frozenset	Colectii care contin multimi unice de date

Python este un limbaj cu **tipizare dinamica** - tipul datelor nu trebuie declarat odata cu acestea cum se făcea în C.

Python este insa (spre deosebire de javascript sau php) un limbaj cu tipizare puternica - odata o variabila declarata sir nu poate fi inclusa în operații aritmetice tipice numerelor întregi

Tipizare în diferite limbaje de programare

```python
a = "vasile"
b = 4.6565

c = a + b

print(type(a))
```

```
Traceback (most recent call last):
  File "/home/main.py", line 7, in
<module>
    c = a + b
TypeError: can only concatenate str (not
"float") to str
```

```php
$a = "vasile";
$b = 4.6565;

$c = $a + $b;

echo($c);
```

```
PHP Warning:  A non-numeric value
encountered in /home/main.php on line 7

4.6565
```

```js
a = "vasile";
b = 4.6565;

c = a + b;
print(c);
```

```
vasile4.6565
```

```perl
$a = "vasile";
$b = 4.6565;


$c = $a + $b;
print($c);
```

```
4.6565
```

Toate variabilele sunt obiecte cu propriile metode

```
print(dir(3)) (3 este numar intreg)
```

['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '__delattr__', '__dir__', '__divmod__', '__doc__', '__eq__', '__float__', '__floor__', '__floordiv__', '__format__', '__ge__', '__getattribute__', '__getnewargs__', '__gt__', '__hash__', '__index__', '__init__', '__init_subclass__', '__int__', '__invert__', '__le__', '__lshift__', '__lt__', '__mod__', '__mul__', '__ne__', '__neg__', '__new__', '__or__', '__pos__', '__pow__', '__radd__', '__rand__', '__rdivmod__', '__reduce__', '__reduce_ex__', '__repr__', '__rfloordiv__', '__rlshift__', '__rmod__', '__rmul__', '__ror__', '__round__', '__rpow__', '__rrshift__', '__rshift__', '__rsub__', '__rtruediv__', '__rxor__', '__setattr__', '__sizeof__', '__str__', '__sub__', '__subclasshook__', '__truediv__', '__trunc__', '__xor__', **'as_integer_ratio'**, **'bit_count'**, **'bit_length'**, **'conjugate'**, **'denominator'**, **'from_bytes'**, **'imag'**, **'numerator'**, **'real'**, **'to_bytes'**]

```
print(dir("3")) (3 este șir de caractere)
```

['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__getnewargs__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmod__', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', **'capitalize', 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isascii', 'isdecimal', 'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition', 'removeprefix', 'removesuffix', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill'**]

Colecții

```
names = ['Ionel', 'Vasilica', 'Pompiliu', 'George', 'Violeta', 'Maria']
names2 = [s for s in names if 'a' in s]
print(names2)
```

```
['Vasilica', 'Violeta', 'Maria']
```

Liste

```
print(dir(names2))
```

['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__gt__', '__hash__', '__iadd__', '__imul__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook__', **'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort'**]

```
dict1 = {'a': 10, 'b': 20, 'c': 30, 'd': 30, 'h': 30}
dict2 = {'a': 20, 'f': 12, 'c': 30, 'j': 30, 'm': 25}

merged_dict = {key: dict1.get(key, 0) + dict2.get(key, 0) for key in set(dict1) | set(dict2)}

print(merged_dict)
```

```
{'m': 25, 'c': 60, 'a': 30, 'd': 30, 'f': 12, 'j': 30, 'b': 20, 'h': 30}
```

Dicționare

```
print(dir(dict1))
```

['__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__ior__', '__iter__', '__le__', '__len__', '__lt__', '__ne__', '__new__', '__or__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__ror__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook__', **'clear', 'copy', 'fromkeys', 'get', 'items', 'keys', 'pop', 'popitem', 'setdefault', 'update', 'values'**]

Operatori

Operatori aritmetici	
Operator	Descriere
+	Adunare
-	Scadere
*	Inmultire
/	Impartire (float)
%	Modulo
**	Exponent
//	Imparțire (int)

Operatori binari	
Operator	Descriere
&	Binary AND
|	Binary OR
^	Binary XOR
~	Ones complement
<<	Binary left shift
>>	Binary right shift

Operatori de comparatie	
Operator	Descriere
==	Egal
!=	Diferit
>	Mai mare
<	Mai mic
>=	Mai mare sau egal
<=	Mai mic sau egal

Operatori logici	
Operator	Descriere
and	Logical AND
or	Logical OR
not	Logical NOT

Nu exista ++ sau - -

Operatori de atribuire	
Operator	Descriere
=	Operator de atribuire
+=	Atribuire cu adunare
-=	Atribuire cu scadere
*=	Atribuire cu inmultire
/=	Atribuire cu impartire (float)
%=	Atribuire cu modulo
**=	Atribuire cu exponent
//=	Atribuire cu impartire (int)

Operatori de incluziune	
Operator	Descriere
in	variabila in secventa
not in	variabila nu e in secventa

Operatori de identitate	
Operator	Descriere
is	true daca variabilele comparate sunt referinte ale aceluias obiect
is not	true daca variabilele comparate nu sunt referinte ale aceluias obiec

Indentarea

```
import random

def bubble_sort(seq):
    """
        Functie care sorteaza o lista
        prin metoda bubble sort
    """

    changed = True
    while changed:
        changed = False
        for i in range(len(seq) - 1):
            if seq[i] > seq[i+1]:
                seq[i], seq[i+1] = seq[i+1], seq[i]
                changed = True
    return seq


n = 10

#generam aleator o lista de n elemente

lista = [random.randrange(1, 100, n) for i in range(n)]

print(lista)
bubble_sort(lista)
print(lista)
```

https://onlinegdb.com/Dg7EkDlta

Parserul Python se foloseste de indentare pentru a detecta blocurile de cod

Se prefera indentarea cu ajutorul spatiilor - cel mai frecvent 4 spatii.

Indentarea poate fi validă sintactic dar greșită logic - poate sa scoata o instrucțiune dintr-o bucla for sau if.

Functii predefinite

Python ofera o serie de funcții predefinite care sunt disponibile tot timpul.

A
abs()
aiter()
all()
any()
anext()
ascii()

B
bin()
bool()
breakpoint()
bytearray()
bytes()

C
callable()
chr()
classmethod()
compile()
complex()

D
delattr()
dict()
dir()
divmod()

E
enumerate()
eval()
exec()

F
filter()
float()
format()
frozenset()

G
getattr()
globals()

H
hasattr()
hash()
help()
hex()

I
id()
input()
int()
isinstance()
issubclass()
iter()

L
len()
list()
locals()

M
map()
max()
memoryview()
min()

N
next()

O
object()
oct()
open()
ord()

P
pow()
print()
property()

R
range()
repr()
reversed()
round()

S
set()
setattr()
slice()
sorted()
staticmethod()
str()
sum()
super()

T
tuple()
type()

V
vars()

Z
zip()

_
__import__()

WOULD YOU LIKE TO KNOW MORE?

https://github.com/pamoroso/free-python-books

O lista de carti despre python care sunt disponibile online

- **Oral History of Guido van Rossum (part1 and part2)**

  Oral history este un program al CHM (Computer History Museum) din Mountain View, California. Cele doua materiale video conțin un interviu extins despre apariția și evoluția Python din punct de vedere al inventatorului acestuia, Guido van Rossum

  https://www.youtube.com/watch?v=Pzkdci2HDpU

- **When Python Practices Go Wrong - Brandon Rhodes**

  Tehnici de programare frecvente dar nu teribil de bune.

  https://www.youtube.com/watch?v=S0No2zSJmks

- **CPython internals and the VM**

  O prezentare a procesului de compilare si interpretare a Python de catre CPython.

  https://www.youtube.com/watch?v=cUyMnGeZ_3c

- **Essentials of Interpretation (incomplet dar intreresant)**

  Detalii despre interpretare și compilare într-un limbaj nu foarte complicat

  https://www.youtube.com/playlist?list=PLGNbPb3dQJ_4WT_m3aI3T2LRf2R_FKM2k

quit()